POLITECNICO
MILANO 1863

Numerical Simulation of Poroelasticity in Bone Tissue

Authors: Matteo Lepidi, Giorgia Bianchi

Course: Advanced Programming for Scientific Computing

Supervisor: Prof. Anna Scotti

Academic Year: 2024/2025

April 5, 2026

Abstract

Understanding how mechanical loads drive interstitial fluid movement in bone is key to explaining the signals that regulate remodeling and skeletal adaptation. Since measuring these microscopic flows directly is extremely difficult [5, 20], computational models become essential tools.

In this work, we developed a three-dimensional finite-element framework based on Biot's theory of poroelasticity, utilizing the "Russian Doll" model of nested porosities [11]. Material parameters were selected from experimental literature on cortical bone [7, 4].

We idealized the cortical bone segment as a hollow cylinder composed of two coupled poroelastic domains: the vascular porosity (PV) and the lacuno-canalicular porosity (PLC). For the discretization, we employed mixed Raviart–Thomas/constant (RT0–P0) finite elements to ensure stability [16, 2].

The core of this report describes the design and implementation of a modular C++ code built upon the `GetFEM++` library. The project specifically tackles the multi-scale coupling between the macro-scale (PV) and the micro-scale (PLC).

We followed Object-Oriented Programming (OOP) principles to keep the code modular and extensible. The architecture clearly separates the physics definitions (`DarcyProblemT`, `ElastProblem`) from mesh management and linear algebra.

Key technical features include:

(i) the use of GetFEM's Generic Weak Form Language (GWFL) to compile weak forms at runtime;

(ii) a custom `InterpolationManager` class that uses an Ordinary Least Squares (OLS) algorithm to transfer data between non-conforming meshes via polynomial fitting;

(iii) the use of Nitsche's method for the weak enforcement of Dirichlet boundary conditions, preserving the symmetry of the monolithic linear system.

We detail the "Russian Doll" algorithm implementation as a one-way coupling strategy: the macroscopic solution drives the microscopic problem through dynamically updated boundary conditions and semi-implicit source terms.

The solver relies on `GMM++` for sparse linear algebra and interfaces with `SuperLU` for the system resolution.

Numerical results demonstrate the software's ability to reproduce physiological pressure gradients and fluid velocities, validating our engineering choices for this multiphysics problem. Specifically, the simulations show that external loads induce significant fluid exchange between PV and PLC. These high-flow regions seem to correspond with sites of bone formation, supporting the hypothesis that fluid flow in the lacuno-canalicular network is a primary stimulus for mechanotransduction [3, 15].

By linking external mechanical loads to microscopic fluid dynamics, this work contributes to a clearer understanding of how mechanics regulate bone remodeling.

The complete source code is available on GitHub: `https://github.com/mlepidus/project-bone-poroelasticity`.

Contents

1 Introduction

Bone is a living porous material that plays a fundamental role in locomotion and structural support. It is continuously subjected to mechanical loads arising from daily activities and exceptional events, such as trauma. Experimental evidence suggests that bone adapts its architecture and density to the applied loads, a process known as bone remodeling [21, 9]. A central mechanism underlying remodeling is the movement of interstitial fluid within the lacuno–canalicular porosity (PLC), where osteocytes reside, and the shear stresses generated by this fluid flow are widely considered a primary mechanotransduction signal [20, 10]. Direct experimental measurement of pressure and velocity in the PLC, however, remains extremely challenging because of the nanometric size of the pores, thus computational poroelastic models have become essential for estimating these quantities and for exploring how fluid flow contributes to bone mechanobiology [5, 4].
In this work, we adapt a numerical poroelastic framework that was originally developed for fluid transport in a *two–dimensional square domain* representing porous rock to the context of cortical bone. Specifically, we implement the hierarchical *Russian Doll* model [11, 22], which captures coupled fluid exchange between the vascular porosity (PV) and the lacuno–canalicular porosity (PLC), and extend it to a three–dimensional hollow cylindrical geometry representing a bone segment. Our objective is to compute the fully coupled mechanical and hydraulic response of these two pore networks under physiologically relevant mechanical loading.

2 Mathematical model

2.1 Poroelasticity

Poroelasticity describes the mechanical behavior of a porous solid saturated with fluid, where deformation of the solid matrix and flow of the fluid are strongly coupled. The theory was first formalized by Biot, introducing a constitutive relation between stress, strain, and pore pressure.
For a poroelastic continuum, the equilibrium equation reads:

$$\nabla \cdot \boldsymbol{\sigma} + \mathbf{F} = 0, \tag{1}$$

where $\boldsymbol{\sigma}$ is the total stress tensor and $\mathbf{F}$ represents external body forces.
The constitutive relation is:

$$\boldsymbol{\sigma} = \lambda \,\mathrm{tr}(\varepsilon(\mathbf{u}))\,\mathbf{I} + \mu\,\varepsilon(\mathbf{u}) - \alpha p \mathbf{I}, \tag{2}$$

where $\mathbf{u}$ is the displacement field, $\varepsilon(\mathbf{u})$ the strain tensor, λ and μ the Lamé parameters, p the pore pressure, and α the Biot coefficient.
Fluid flow in the porous medium is governed by Darcy's law:

$$\mathbf{q} = -\frac{\kappa}{\eta}\nabla p, \tag{3}$$

with κ the permeability tensor, η the fluid viscosity, and $\mathbf{q}$ the percolation velocity.

2.2 Hierarchical Porosity in Bone

Cortical bone exhibits a hierarchical pore network:

- **Vascular porosity (PV):** channels containing blood vessels (Haversian and Volkmann canals), with typical pore size ~ 50 μm.
- **Lacuno-canalicular porosity (PLC):** the network embedding osteocytes and their canaliculi, with characteristic pore size ~ 100 nm.
- **Collagen-apatite porosity (PCA):** spaces between mineral and collagen (~ 1 nm), usually negligible in terms of fluid transport.

Since PCA fluid flow is negligible, we will focus on PV and PLC. The size difference between the two porosities is approximately two orders of magnitude [7].

2.3 Russian Doll Model

The Russian Doll poroelastic model [7] considers each porosity level as an independent poroelastic continuum. Fluid exchange between levels occurs via leakage terms proportional to the pressure difference, which means that in cortical bone PV and PLC are modeled separately, but coupled through fluid transfer:

$$\dot{m} = \gamma(p_v - p_l),$$

where γ is the leakage coefficient, p_v the vascular pressure, and p_l the lacuno-canalicular pressure.

It is important to note that for the resolution of our problem, since the scale of the PV problem is much greater compared to the one of the PLC problem, we can assume that the effect of the smaller system on the bigger one is negligible. In practice this means that the p_l term in the equations for vascular porosity problem can be set to 0.
Moreover, the boundary conditions for the PLC problem are determined through the resolution of the vascular model. In particular the pressure from the PV system is applied in the Haversian canal of the osteon, while the displacement is applied on the external surface.

2.4 Governing Equations

2.4.1 Vascular Porosity (PV)

The governing equations are:

$$\nabla \cdot \boldsymbol{\sigma}_v + \mathbf{F}_v = 0, \tag{4}$$

$$\boldsymbol{\sigma}_v = \lambda_v \operatorname{tr}(\varepsilon(\mathbf{u}_v))\,\mathbf{I} + \mu_v\,\varepsilon(\mathbf{u}_v) - \alpha_v p_v \mathbf{I}, \tag{5}$$

$$\frac{1}{M_v}\frac{\partial p_v}{\partial t} - \kappa_v \nabla^2 p_v + \frac{\partial}{\partial t}\operatorname{tr}(\alpha_v \varepsilon(\mathbf{u}_v)) = -\gamma(p_v - p_l), \tag{6}$$

where:

- $\mathbf{u}_v$: displacement field,
- p_v: pore pressure in PV,
- λ_v, μ_v: Lamé parameters,
- M_v: Biot modulus,
- κ_v: permeability of PV,
- γ: leakage coefficient.

2.4.2 Lacuno-Canalicular Porosity (PLC)

Analogously, the PLC domain is governed by:

$$\nabla \cdot \boldsymbol{\sigma}_l + \mathbf{F}_l = 0, \tag{7}$$

$$\boldsymbol{\sigma}_l = \lambda_l \operatorname{tr}(\varepsilon(\mathbf{u}_l))\,\mathbf{I} + \mu_l\,\varepsilon(\mathbf{u}_l) - \alpha_l p_l \mathbf{I}, \tag{8}$$

$$\frac{1}{M_l}\frac{\partial p_l}{\partial t} - \kappa_l \nabla^2 p_l + \frac{\partial}{\partial t}\operatorname{tr}(\alpha_l \varepsilon(\mathbf{u}_l)) = \gamma(p_v - p_l), \tag{9}$$

where p_l is the PLC pore pressure, κ_l its permeability, and M_l the Biot modulus of PLC.

2.5 Boundary Conditions

To close the system of Partial Differential Equations, appropriate boundary conditions must be specified for both the fluid and solid phases on the domain boundary $\Gamma = \partial\Omega$. Let Γ_D and Γ_N denote the Dirichlet and Neumann boundaries, respectively, such that $\Gamma_D \cup \Gamma_N = \Gamma$ and $\Gamma_D \cap \Gamma_N = \emptyset$.

For the fluid phase (Darcy problem), we impose:

$$p = p_D \quad \text{on } \Gamma_D^f, \tag{10}$$

$$-\frac{\kappa}{\eta}\nabla p \cdot \mathbf{n} = g_N \quad \text{on } \Gamma_N^f, \tag{11}$$

where p_D is the prescribed pressure and g_N is the normal fluid flux.

For the solid phase (Elasticity problem), the conditions are:

$$\mathbf{u} = \mathbf{u}_D \quad \text{on } \Gamma_D^s, \tag{12}$$

$$\boldsymbol{\sigma}\mathbf{n} = \mathbf{t} \quad \text{on } \Gamma_N^s, \tag{13}$$

where $\mathbf{u}_D$ is the prescribed displacement and $\mathbf{t}$ is the surface traction vector.

In the specific context of the Russian Doll model:

- For the **Vascular Porosity (PV)** problem, the boundary conditions are derived from the external loads applied to the bone segment (e.g., axial compression or torsion).
- For the **Lacuno-Canalicular Porosity (PLC)** problem, the boundary conditions on the outer wall of the osteon are not arbitrary but are determined by interpolating the macroscopic solution $(p_v, \mathbf{u}_v)$ obtained from the PV problem. This creates the hierarchical one-way coupling characteristic of the multiscale approach.

3 Numerical Discretization

3.1 Finite Element Discretization

The governing equations are discretized with a mixed finite element method in order to capture both velocity and pressure unknowns in a stable way.

Raviart–Thomas spaces. The velocity field is discretized with Raviart–Thomas elements of lowest order (RT_0). These finite element spaces are specifically designed for Darcy-type problems, since they enforce continuity of the normal component of the velocity across element interfaces. This guarantees local conservation of mass at the discrete level, which is crucial in poroelasticity. On each tetrahedron K, the space is defined as

$$\mathrm{RT}_0(K) = \left\{ \mathbf{v}(x) = \mathbf{a} + b\,x \ : \ \mathbf{a} \in \mathbb{R}^3,\ b \in \mathbb{R} \right\},$$

This choice leads to piecewise affine velocities with constant divergence inside each element [16, 2].

Pressure and coefficients. The pressure is discretized with piecewise constant elements ($\mathbb{P}_0$), a natural pairing with RT_0 that satisfies the inf–sup stability condition. Material coefficients and poroelastic parameters are also discretized with $\mathbb{P}_0$ functions, while viscosity terms are approximated with linear $\mathbb{P}_1$ elements, i.e. `FEM_PK(3,1)`.

Integration. All integrals are evaluated using second-order tetrahedral quadrature rules (`IM_TETRAHEDRON(2)`), which provide a good compromise between accuracy and computational cost.

Summary. The resulting mixed finite element scheme can be summarized as:

- Velocity: RT_0 (`FEM_RT0(3)`), ensuring flux conservation;
- Pressure and coefficients: $\mathbb{P}_0$ (`FEM_PK(3,0)`);
- Viscosity-related terms: $\mathbb{P}_1$ (`FEM_PK(3,1)`);
- Numerical integration: second-order tetrahedral quadrature.

This combination yields a stable and efficient discretization for large-scale three-dimensional poroelastic simulations.

3.2 Time Discretization

We adopt an implicit Backward Euler scheme for temporal discretization. This allows for unconditional stability, which is essential given the varying time scales of fluid diffusion and solid deformation. The fully discrete system results in a monolithic linear system solved at each time step t^{n+1}.

3.3 Algebraic Form

After finite element discretization, the PDE systems of the vascular (PV) and lacuno-canalicular (PLC) porosities are transformed into algebraic equations. Following [18], the resulting discrete problems at each time step N are:

Vascular porosity (PV):

$$Ku_v^N - \alpha_v C p_v^N = R_v^N, \tag{14}$$

$$\alpha_v C^T \dot{u}_v^N + (H_v + \gamma M) p_v^N + \frac{1}{M_v} M \dot{p}_v^N = Q_v^N + \gamma M p_l^N, \tag{15}$$

Lacuno-canalicular porosity (PLC):

$$Ku_l^N - \alpha_l C p_l^N = R_l^N, \tag{16}$$

$$\alpha_l C^T \dot{u}_l^N + (H_l + \gamma M) p_l^N + \frac{1}{M_l} M \dot{p}_l^N = Q_l^N + \gamma M p_v^N, \tag{17}$$

where K is the stiffness matrix, C the coupling matrix, H_n the hydraulic conductivity matrix for porosity $n \in \{v, l\}$, M the mass matrix, and R_n, Q_n the external load vectors. The coupling between PV and PLC appears through the leakage term $\gamma(p_v - p_l)$.
The matrices above are defined as follows:

$$K = \int_V B_u^T D B_u \, dV, \qquad H_n = \int_V B_p^T K_n B_p \, dV,$$

$$C = \int_V B_u N \, dV, \qquad M = \int_V N^T N \, dV,$$

$$R_n = \int_V N^T F_n \, dV + \int_S N^T T \, dS, \qquad Q_n = \int_S N^T q_n \, dS,$$

with $n \in \{v, l\}$. Here, N is the matrix of shape functions, B_u and B_p are the deformation matrices expressed in terms of derivatives of the shape functions, D is the constitutive stress–strain matrix, and K_v, K_l are the permeability matrices for PV and PLC, respectively. F_n is the external volume load, T is the external surface load, and q_n is the fluid flux vector.

By assembling the unknowns of displacements u, fluxes q, and pressures p together, and applying the backward Euler scheme for time discretization, the problem becomes the following block matrix system:

$$\begin{bmatrix} K & 0 & -\alpha D^T \\ 0 & A & -B^T \\ \frac{\alpha D}{\Delta t} & B & M\left(\frac{1}{\Delta t} + \gamma\right) \end{bmatrix} \begin{bmatrix} u^{n+1} \\ q^{n+1} \\ p^{n+1} \end{bmatrix} = \begin{bmatrix} F + F^{BC} \\ p^{BC} \\ \gamma M p_l + \frac{M}{\Delta t} p^n + \frac{\alpha D}{\Delta t} u^n \end{bmatrix}. \tag{18}$$

This formulation highlights the dual-porosity structure of the Russian Doll model: two poroelastic problems with the same algebraic structure, coupled only by the leakage operator. In our implementation, the PV system is solved first, and its solution provides the boundary data for the PLC system.

3.4 Multiscale Coupling Algorithm

To solve the coupled multiscale problem, we adopt a sequential scheme that exploits the hierarchical nature of the bone porosity: since the micro-scale domain (PLC) is orders of magnitude smaller than the macro-scale domain (PV), we assume a one-way coupling where the macroscopic solution drives the microscopic behavior, while the feedback from the micro to the macro scale is considered negligible. The solution procedure advances in two steps at each time level t^{n+1}.

Macroscopic Step (PV). The process begins by solving the poroelastic system on the macroscopic vascular porosity (PV) domain. At this stage, the coupling term dependent on the microscopic pressure p_l is decoupled, allowing the PV problem to be solved as a standard poroelastic problem with appropriate physiological boundary conditions.

Microscopic Step (PLC). Once the macroscopic solution (displacement $\mathbf{u}_v$ and pressure p_v) is available, it must be transferred to the microscopic domain to serve as boundary conditions.
Given the vast difference in scale, the macroscopic field variations along the transverse directions of the single osteon are negligible. Therefore, we approximate the boundary data for the PLC problem as a one-dimensional function of the longitudinal coordinate z only.
Mathematically, this is achieved by extracting the discrete macroscopic solution along the osteon axis and reconstructing a continuous profile via an *Ordinary Least Squares (OLS)* regression. This yields smooth polynomial functions $P(z)$ and $\mathbf{U}(z)$, which filter out local numerical noise from the coarse macro-mesh. These polynomials are then imposed as Dirichlet boundary conditions on the outer surface of the PLC domain.
Finally, the microscopic problem is solved including the leakage source term, which is now fully determined by the interpolated macroscopic pressure.

4 Software Architecture and Design

The computational core of the project is designed as a C++ code based on the `GetFEM++` library [17]. The architecture isolates the definitions of physical problems from the mesh management and linear algebra implementations.

4.1 Overview of the Code Structure

The code follows a hierarchical structure orchestrated by a main driver (`main_coupled.cc`). The architecture can be divided into three logical layers:

- **Data Layer:** handled by classes like `Bulk`, `BulkDarcyData`, and `BulkElastData`. These classes are responsible for parsing input files via `GetPot`, managing the computational mesh (generation or import), and storing physical parameters.
- **Physics Layer:** handled by `DarcyProblemT` and `ElastProblem`. These classes define the Finite Element spaces, manage Degree of Freedom (DOF) mappings, and

handle the assembly of local stiffness and mass matrices using GetFEM's generic assembly language.

- **Orchestration Layer:** the `CoupledProblem` and `RussianDollProblem` classes manage the interaction between different physics. `CoupledProblem` builds the monolithic linear system for poroelasticity, while `RussianDollProblem` manages the one-way coupling between the macro-scale (Vascular Porosity) and micro-scale (Lacuno-Canalicular Porosity).

The project follows a modular architecture:

```
project-bone-poroelasticity/
 include/            # Header files (.h)
    Core.h          # Type definitions, includes
    Bulk.h          # Domain management
    FEM.h           # Finite element wrapper
    LinearSystem.h # Linear algebra interface
    CoupledProblem.h # Main problem class
    ...
 src/                # Implementation files (.cc)
 meshes/             # Mesh files (Gmsh format)
 main_coupled.cc     # Main driver programs
 main_russian_doll.cc
 Makefile            # Build configuration
```

In Table 1 an overview of the external dependencies:

Table 1: External Dependencies

Library	**Purpose**	**Version**
GetFEM++	Finite element framework	5.4.4
GMM++	Sparse linear algebra	(bundled with GetFEM)
SuperLU	Direct sparse solver	System
Boost	Smart pointers, Spirit parser	1.7x+
GetPot	Parameter file parsing	Included

4.2 Design Patterns and Principles

The software architecture is built on standard Object-Oriented principles to ensure the code is modular and facilitate future extensions. Specifically, we focused on the following design choices:

Encapsulation and Separation of Concerns. We organized the code so that each class handles a specific task. For instance, the `Bulk` class acts as a container for the mesh and physical parameters, keeping data management separate from the numerical solving process. Similarly, the `TimeLoop` class handles the logic for time discretization, which keeps the main solver loop clean and focused only on the solution steps.

Composition over Inheritance. To avoid complex class hierarchies, we used composition for the multiphysics problem. The `CoupledProblem` class simply holds pointers to the sub-problems (`M_ElastPB` and `M_DarcyPB`), allowing the coupled solver to manage the interaction between the solid and fluid mechanics without being strictly tied to how those solvers are implemented internally.

Inheritance for Specialization. We used inheritance only when it offered a clear advantage in reusing code. Since the microscopic problem is mathematically very similar to the macroscopic one, the `PLCProblem` class inherits directly from `CoupledProblem`. This allowed us to reuse the existing code for assembling the matrices, while only overriding specific functions to add the leakage term (γ) and the source term needed for the dual-porosity coupling.

Memory Management. Since 3D simulations require significant memory, we managed large data structures (like the sparse matrices in `LinearSystem`) using smart pointers (such as `boost::shared_ptr`). This ensures that memory is automatically freed when it is no longer needed, preventing leaks during long simulations.

4.2.1 Class Hierarchy

Here is a brief overview of the class hierarchy. A more detailed explanation is provided in section 6.2.

Bulk. The `Bulk` class acts as the main container for the simulation's geometry and physics. It manages the mesh generation, either by creating a structured grid or importing a file from Gmsh, and stores all material properties using the `BulkDarcyData` and `BulkElastData` objects.

FEM. We designed this class to simplify the usage of the `getfem::mesh_fem` object. It handles the setup of specific finite element spaces (such as Raviart-Thomas for velocity) and provides useful helper methods to retrieve Degree of Freedom (DOF) coordinates.

LinearSystem. This class manages all linear algebra operations. It wraps the `gmm::row_matrix` sparse format and provides a generic interface for solver configuration. This allows the rest of the code to remain unchanged regardless of whether we use a direct solver (like SuperLU) or an iterative one (like GMRES), effectively separating the numerical method from the solver backend.

DarcyProblemT & ElastProblem. These classes define the weak formulations for the two physical problems. `DarcyProblemT` handles the mixed formulation for the fluid flow, while `ElastProblem` deals with the displacement-based elasticity. Their main job is to assemble the local stiffness and mass matrices and apply the relevant boundary conditions.

CoupledProblem & PLCProblem. The `CoupledProblem` class is responsible for assembling the global monolithic block matrix that links Darcy and Elasticity. `PLCProblem` inherits from it and adapts the formulation for the micro-scale porosity, specifically by adding the leakage terms required by the Russian Doll model to the Right-Hand Side.

RussianDollProblem. This class controls the entire multiscale simulation workflow. It holds the two separate problem instances (the Vascular Porosity PV and the Lacuno-Canalicular Porosity PLC) and manages the solution sequence. It uses the `InterpolationManager` to transfer data between scales, ensuring that the macroscopic solution correctly updates the boundary conditions and source terms of the microscopic problem.

InterpolationManager. The `InterpolationManager` handles data transfer between the meshes. It implements two strategies: line interpolation, which extracts data along a 1D line to fit polynomials, and mesh interpolation, which performs a direct 3D-to-3D projection using GetFEM's interpolation functions.

4.2.2 Data Flow

The simulation workflow begins with the `GetPot` parser reading the configuration file. This raw data initializes the `Bulk` object, which constructs the computational domain. Subsequently, `FEM` objects are instantiated on this mesh to define the discrete functional spaces.
During the initialization phase, the problem classes (`DarcyProblemT`, `ElastProblem`) use these spaces to pre-allocate the sparse matrix structures and perform the initial assembly of the linear system matrix, which remains constant for linear problems. The execution then enters the time loop managed by `TimeLoop`.
At each time step:

1. The `CoupledProblem` coordinates the assembly of the time-dependent Right-Hand Side (RHS) and applies the current boundary conditions, incorporating contributions from the sub-problems.

2. The `LinearSystem` solves the algebraic system $\mathcal{A}x = b$ (reusing the pre-assembled matrix $\mathcal{A}$).

3. The solution vector is distributed back to the physics classes to update their internal state.

4. Finally, the results are exported to VTK format using the wrappers in `UsefulFunctions`, making the data ready for post-processing in ParaView.

In the specific case of the Russian Doll simulation, this cycle is nested: the PV problem is solved first, its solution is processed by the interpolation layer, and the resulting coefficients are used to configure and solve the PLC problem before moving to the next time step.

5 Numerical Implementation

This section details the translation of the numerical models described in Chapter 3 into the C++ software architecture. The implementation leverages the `GetFEM++` library for finite element management and `GMM++` for sparse linear algebra, wrapped in high-level classes to separate the physical logic from the numerical backend.

5.1 Finite Element Space Management

While Chapter 3 defined the theoretical spaces (RT_0, P_0, P_1), their instantiation in the software is handled dynamically by the `FEM` wrapper class.

- **Dynamic Initialization:** Instead of hardcoding the element types, the `FEM` constructor parses the input file (via `GetPot`) to select the appropriate descriptor (e.g., `"FEM_RT0(3)"` or `"FEM_PK(3,1)"`). This allows users to switch between different discretization orders at runtime without recompiling.
- **Vectorial Fields:** The class manages the *quotient dimension* ($qdim$) of the finite element space based on the variable type. By checking the variable tag (e.g., "Velocity" or "Displacement"), it automatically sets the vector dimension ($qdim = 3$), ensuring the correct allocation of Degrees of Freedom (DOFs) for vector fields versus scalar fields (Pressure).

5.2 Matrix Assembly

A key feature of the implementation is the use of GetFEM's Generic Weak Form Language (GWFL). Instead of implementing manual loops over quadrature points, which are error-prone and verbose, the weak forms are expressed as strings parsed at runtime.

5.2.1 Darcy Operators Implementation

Recall that the discrete algebraic system takes the form of a saddle-point problem:

$$\begin{bmatrix} A_{qq} & B^T \\ B & -\left(\frac{1}{M\Delta t}M_{pp} + \gamma M_{leak}\right) \end{bmatrix} \begin{bmatrix} \mathbf{q}^{n+1} \\ p^{n+1} \end{bmatrix} = \begin{bmatrix} \mathbf{f}_q \\ f_p + \frac{1}{M\Delta t}M_{pp}p^n \end{bmatrix} \tag{19}$$

The assembly of the mixed Darcy system is encapsulated in `DarcyOperatorsBulk.cc` as follows:

- **Velocity Mass Matrix:** $A_{qq} = \int_\Omega \mathbf{K}^{-1}\mathbf{v} \cdot \mathbf{w}\, d\Omega$ is the mass matrix for velocity weighted by the inverse permeability tensor. The term $\int_\Omega \mathbf{K}^{-1}\mathbf{q} \cdot \mathbf{v}$ is assembled by the string:

```
assem.set("M(#1,#1)+="
          "comp(vBase(#1).vBase(#1).Base(#2))(:,i,:,i,j).invK(j)");
```

- **Divergence Constraint:** $B = -\int_\Omega (\nabla \cdot \mathbf{v})\phi\, d\Omega$ represents the divergence constraint. The coupling term $-\int_\Omega p\nabla \cdot \mathbf{v}$ is implemented in the `divHdiv` function as:

```
assem.set("M(#1,#2)+=-comp(vGrad(#1).Base(#2))(:,i,i,:)");
```

This command instructs the library to compute the contraction between the divergence of the velocity shape functions (`vGrad`) and the pressure basis functions.

- M_{pp} is the standard mass matrix for pressure, used for the time accumulation term.

The time-dependent nature of the problem is handled using a **backward Euler implicit scheme**. In the code, this is implemented by scaling the pressure mass matrix M_{pp} by $1/\Delta t$ and adding it to the diagonal of the system.

5.2.2 Elasticity Stiffness Implementation

The mechanical problem is governed by the momentum balance equation. We employ a standard Galerkin formulation for the displacement field $\mathbf{u}$. The local stiffness matrix is assembled in `ElastOperatorsBulk.cc` via the `stiffElast` function, which integrates the weak form of the constitutive law:

$$\int_\Omega \sigma(\mathbf{u}) : \varepsilon(\mathbf{v})\, d\Omega = \int_\Omega (\lambda(\nabla \cdot \mathbf{u})(\nabla \cdot \mathbf{v}) + 2\mu\, \varepsilon(\mathbf{u}) : \varepsilon(\mathbf{v}))\, d\Omega \tag{20}$$

where λ and μ are the Lamé parameters. The Right-Hand Side (RHS) includes contributions from body forces (gravity) and surface tractions (Neumann boundaries), assembled by `stressRHS`.
So the `ElastOperatorsBulk::stiffElast` function assembles the mechanical stiffness matrix. The constitutive law for linear elasticity is expressed directly in the GWFL, utilizing the Lamé parameters λ and μ interpolated from the data file:

```
assem.set("lambda=data$1(#2);mu=data$2(#2);"
          "t=comp(vGrad(#1).vGrad(#1).Base(#2));"
          "M(#1,#1)+=␣sym(t(:,i,j,:,i,j,k).mu(k)"
          "+t(:,j,i,:,i,j,k).mu(k)"
          "+t(:,i,i,:,j,j,k).lambda(k))");
```

5.3 Monolithic System Assembly

The `CoupledProblem` class is responsible for aggregating the local matrices above into the global monolithic system described in Chapter 3.
The global system is constructed by the `CoupledProblem` class, which aggregates the sub-matrices from the Darcy and Elasticity problems into a single block structure:

$$\mathcal{A}_{global} = \begin{bmatrix} K_{uu} & 0 & C_{up} \\ 0 & A_{qq} & B^T \\ C_{pu} & B & A^*_{pp} \end{bmatrix} \tag{21}$$

The off-diagonal blocks C_{up} and C_{pu} represent the Biot coupling term $\alpha \nabla \cdot \mathbf{u}$.

- **Block Construction:** The global matrix is built using the `LinearSystem::addSubMatrix` method, which inserts the sub-matrices (Elasticity, Darcy, Coupling) at specific offsets. For instance, the Darcy block starts at row/-column index N_u, where N_u is the number of displacement DOFs.

- **Off-Diagonal Coupling:** The poroelastic coupling term $C_{pu} = \int \alpha(\nabla \cdot \mathbf{u})p$ is explicitly assembled by the `matrixFluidP` function. Since this term links the displacement space (P_1) with the pressure space (P_0), it creates the rectangular off-diagonal blocks necessary for the monolithic solver.

Summary of the process:

1: **Initialize** empty global sparse matrix $\mathcal{A}$
2: **Darcy:** Assemble A_{qq}, B, and M_{pp}
3: **Elasticity:** Assemble stiffness K_{uu}
4: **Coupling:** Assemble interaction matrix $C_{pu} = \int \alpha(\nabla \cdot \mathbf{v})p$
5: **Global Build:** Insert sub-matrices into $\mathcal{A}$ at calculated offsets via `LinearSystem::addSubMatrix`

5.4 Boundary Condition Handling

The application of boundary conditions separates the geometric identification of boundaries from the numerical enforcement.

5.4.1 Region Tagging and Parsing

The `BC` class maps geometric boundaries to numerical regions (for more details see section 6.2.2). It supports two modes:

1. **Gmsh Tags:** It reads physical names (e.g., "EndCaps") from the mesh file and maps them to internal integer IDs.

2. **Geometric Detection:** For meshes without tags, it classifies faces based on their normal vectors (e.g., distinguishing inner vs. outer cylindrical surfaces).

The values of the boundary conditions are evaluated using the `Parser` class, which interprets mathematical expressions from the input file (e.g., $P = 10\sin(t)$) at runtime.

5.4.2 Weak Enforcement

- **Neumann Conditions:** Natural boundary conditions (prescribed flux or traction) are integrated directly into the RHS vectors during the assembly phase.

- **Dirichlet Conditions:** The solver supports both strong enforcement (via matrix row/column modification) and weak enforcement via Nitsche's method [13].

Nitsche's Method. For the Darcy velocity and Elasticity displacement, we implemented Nitsche's method in `DarcyOperatorsBD.cc` to enforce Dirichlet conditions weakly. For the elasticity problem, the consistent weak form is augmented with symmetry and penalty terms on the Dirichlet boundary Γ_D:

$$a_{Nitsche}(\boldsymbol{u}, \boldsymbol{v}) = -\int_{\Gamma_D} (\boldsymbol{\sigma}(\boldsymbol{u})\mathbf{n}) \cdot \boldsymbol{v}\, d\Gamma - \int_{\Gamma_D} (\boldsymbol{\sigma}(\boldsymbol{v})\mathbf{n}) \cdot \boldsymbol{u}\, d\Gamma + \sum_{E \in \Gamma_D} \frac{\beta\mu}{h_E} \int_E \boldsymbol{u} \cdot \boldsymbol{v}\, d\Gamma \quad (22)$$

where μ is the shear modulus, h_E is the local element diameter, and $\beta = 5$ is a stabilization parameter chosen to ensure coercivity. A similar formulation is applied to the pressure field in the Darcy problem, as implemented in the `essentialWNitsche` operators.
This method offers significant software engineering advantages: it avoids modifying the sparsity pattern of the global matrix after assembly and preserves the symmetry of the problem, allowing the use of more efficient linear solvers.

5.5 Dual-Porosity Algorithm Implementation (Russian Doll)

The "Russian Doll" coupling strategy, based on the theoretical work of Cowin and Gailani [8, 5] and described in Section 3.4, is implemented in the `RussianDollProblem` class. The solution algorithm at each time step t^n proceeds as follows:

1. **Solve PV problem**: The macroscopic `CoupledProblem` is solved first, with the coupling term temporarily set to zero ($p_l = 0$). The justification for this one-way coupling lies in the scale separation: the PLC domain is orders of magnitude smaller than the PV domain, allowing transverse variations (x, y) to be neglected and boundary data to be approximated as 1D functions.

2. **Extract solution along line**: The algorithm invokes `getfem::interpolation()` to project the computed PV pressure and displacement fields onto a set of sample points distributed along the osteon axis (defined in the input file).

3. **Polynomial fitting**: The `InterpolationManager` class processes the sampled data using an Ordinary Least Squares (OLS) regression. This step converts the discrete raw data into continuous polynomial objects (functors) capable of being evaluated at any coordinate z. For more details, see section 6.2.10.

4. **Update PLC boundaries**: These polynomials are passed to the `PLCProblem`. Specific callback functions are registered to define the Dirichlet values for displacement and the Neumann values for pressure on the outer boundary of the osteon mesh.

5. **Update source terms**: The leakage source term $+\gamma M p_v$ is updated using the fitted polynomial for the macroscopic pressure.

6. **Solve PLC problem**: Finally, the microscopic problem is assembled and solved with the updated boundary conditions and right-hand side.

This implementation strictly separates the physics of the two scales: the `RussianDollProblem` acts as a wrapper that simply moves data from one solver to the other, ensuring that the computationally expensive meshing and assembly operations remain encapsulated within their respective problem classes.

6 Code Implementation Details

Code Availability: The complete, documented source code for this poroelasticity framework, including all classes discussed below, is publicly available on GitHub: `https://github.com/mlepidus/project-bone-poroelasticity`.

The repository contains the full implementation, example configuration files, meshes, and build scripts. The code is extensively commented following Doxygen standards, and the generated documentation can be easily generated through the command `doxygen Doxyfile`.

6.1 Global Type Definitions

To ensure type safety and facilitate potential future changes in precision or linear algebra backends, the code relies on a set of global type definitions located in `Core.h`.

- **Scalar and Size Types:** We define `scalar_type` (typically `double`) for floating-point arithmetic and `size_type` (typically `std::size_t` or `unsigned`) for indexing loops and arrays.
- **Sparse Matrices:** The project leverages `GMM++` for sparse linear algebra. The type `sparseMatrix_Type` is aliased to `gmm::row_matrix<sparseVector_Type>`. This Row-Compressed Storage (CSR-like) format is essential for minimizing memory footprint when assembling large Finite Element stiffness matrices.
- **Smart Pointers:** To manage the lifetime of heavy objects like matrices and solution vectors, we utilize `boost::shared_ptr` (or `std::shared_ptr` in C++11 compliant compilers). This enables RAII (Resource Acquisition Is Initialization), ensuring that the global system matrix is automatically deallocated when the linear system object is destroyed.

Listing 1: Core type definitions

```
typedef gmm::rsvector<scalar_type> sparseVector_Type;
typedef gmm::row_matrix<sparseVector_Type> sparseMatrix_Type;
typedef boost::shared_ptr<sparseMatrix_Type> sparseMatrixPtr_Type;
typedef std::vector<scalar_type> scalarVector_Type;
typedef boost::shared_ptr<scalarVector_Type> scalarVectorPtr_Type;
```

6.2 Main Classes

6.2.1 Bulk Class

The `Bulk` class serves as the central manager for the computational domain, acting as a container for both geometric objects (mesh, regions) and physical properties (materials). It abstracts the complexity of mesh handling, providing a unified interface to the solvers regardless of the underlying mesh source.

Mesh Generation Strategy The class constructor implements a dual approach to mesh initialization based on the configuration file:

- **Internal Generation (Structured):** If no external file is specified, the code generates a structured semi-regular mesh using `getfem::regular_unit_mesh`. This unit mesh is then scaled to the physical dimensions (L_x, L_y, L_z) via a geometric transformation matrix. This mode is primarily used for code verification (e.g., unit cube tests) and debugging.

- **External Import (Unstructured):** For realistic bone geometries, the code imports unstructured tetrahedral meshes generated by Gmsh (.msh files) via the `getfem::import_mesh_gmsh` routine. We provided inside the `./meshes/` folder some ready to use examples.

Physical Region Mapping To facilitate boundary condition application, the `Bulk` class maps physical tags (strings) from the imported Gmsh file to GetFEM's internal integer region IDs. The `M_regmap` container stores these associations, allowing the user to refer to boundaries using semantic names (e.g., "Periosteum", "EndCaps") in the configuration file. The class performs diagnostic checks during initialization to report the existence and element count of each region.

Material Data Encapsulation The class encapsulates two specialized data containers, where we can find specific parameters and strings to be parsed: `BulkDarcyData` for the flow problem (permeability $\mathbf{K}$, viscosity μ, etc.) and `BulkElastData` for mechanics (Lamé parameters, Young's modulus).

Dimension Validation The `Bulk` class ensures consistency between the requested physics and the geometry. It compares the spatial dimension defined in the input file against both the geometric transformation descriptor and the imported mesh, providing detailed error messages to guide users in correcting configuration mismatches.

Listing 2: Bulk class construction and region diagnostics

```
// Constructor with multiple configuration sections
Bulk(const GetPot& dataFile,
     const std::string& section = "bulkData/",
     // ... section definitions
     const std::string& sectionElast = "mecc/");

// Check if external mesh file is specified
if (M_meshFile.compare("none") == 0) {
    // Generate internal structured mesh
    getfem::regular_unit_mesh(M_mesh, nsubdiv, pgt, false);
    M_hasExternalMesh = false;
} else {
    // Import mesh from Gmsh file
    getfem::import_mesh_gmsh(M_meshFolder + M_meshFile, M_mesh, M_regmap);
    M_hasExternalMesh = true;
    size_type meshDim = M_mesh.dim();
if (meshDim != M_dim) {
    throw std::runtime_error("DIMENSION MISMATCH ERROR\n"
                             "Requested dimension: " +
                             std::to_string(M_dim) + "\n"
                             "External mesh dimension: " +
                             std::to_string(meshDim));
}
```

Dual-Porosity Configuration For the Russian Doll coupling, the framework instantiates two separate `Bulk` objects: one for the PV domain and another for the PLC domain. This separation is configured through distinct data file sections (`[bulkDataPV]`

and [bulkDataPLC]), allowing independent control over the resolution of the mesh and the properties of the material. Typically, the PLC domain uses a finer mesh to resolve micro-scale structures, while the PV domain employs a coarser mesh for the macro-scale vascular network.

6.2.2 BC Class

The BC class provides comprehensive boundary condition management for both scalar and vector problems, with specialized extensions to support the Russian Doll coupling requirements.

- **Flexible Boundary Assignment:** The class implements multiple strategies to identify physical boundaries:
  - **Geometric Detection:** Two specialized methods allow automatic detection without external tags. setBoundariesCylinder() analyzes face normals to distinguish inner/outer surfaces in cylindrical geometries, while setBoundariesSquare() uses a Bounding Box approach to identify faces lying on the domain limits $(x_{min}, x_{max}, y_{min}, y_{max})$.
  - **Gmsh Tag Mapping:** Two complementary methods handle Gmsh physical tags: setBoundariesFromTagsName() uses semantic names (e.g., "outer", "top"), mapping them to internal IDs (0-3). Alternatively, setBoundariesFromTagNumbers() works directly with numerical tags, selecting either the largest or smallest tags in sorted order. This provides precise control over boundary assignment for complex unstructured meshes.
- **Expression-Based BC Specification:** Boundary conditions can be specified analytically using an internal expression parser. The class stores expression strings for different BC types (Neumann pressure, Dirichlet displacement, etc.) which are evaluated at runtime using spatial variables (x, y, z), normal ID (n), and time (t).
- **Polynomial Callback System:** For the Russian Doll coupling, the class implements a priority callback mechanism that overrides standard parser evaluation:
  - **Pressure Callbacks:** Scalar functions $p(z)$ specified via setPressureCallback() or polynomial coefficients via setPressurePolynomial(). These are essential for applying the PV pressure profile onto the PLC outer wall.
  - **Displacement Callbacks:** Vector functions $\mathbf{u}(z)$ specified either as a complete vector callback or as separate component callbacks (u_x, u_y, u_z).
  - **Dynamic Override:** When a callback is registered for a specific region, the evaluation methods (e.g., BCNeum()) automatically bypass the parser and execute the callback.

Listing 3: BC class callback interface for Russian Doll coupling

```
// Callback types for polynomial BCs
using ScalarCallback = std::function<scalar_type(scalar_type z)>;
using VectorCallback = std::function<bgeot::base_node(scalar_type z)>;
```

```
// Set polynomial coefficients for pressure BC
void setPressurePolynomial(size_type region,
                           const std::vector<scalar_type>& coefficients,
                           scalar_type z_min, scalar_type z_max);

// Evaluate with callback override strategy
scalar_type BCNeum(const base_node& x, const size_type& flag, const
   scalar_type t) {
    auto it = M_pressureCallbacks.find(flag);
    // Priority check: if callback exists, use it
    if (it != M_pressureCallbacks.end() && it->second) {
        return it->second(x[2]);  // Evaluate polynomial p(z)
    }
    // Fallback: use standard parser evaluation string
    M_parser.setString(M_BCNeum);
    // ... set variables and evaluate
    return M_parser.evaluate();
}
```

Russian Doll Integration: In the dual-porosity framework, the `BC` class acts as the bridge between the PV and PLC problems. After the polynomial fitting of the PV solution, the `RussianDollProblem` calls `setPressurePolynomial()` on the PLC boundary object.

- **Automatic Normalization:** The polynomial evaluation method `evaluatePolynomial()` internally normalizes the z-coordinate to the range $[0, 1]$ using the stored z_{min} and z_{max} values. This ensures numerical stability and consistent evaluation regardless of the physical mesh height.
- **Coefficient Storage:** The class maintains separate containers (e.g., `M_pressurePolynomials`) to store the raw polynomial coefficients. This encapsulation allows the reconstruction of the boundary field at any quadrature point during the assembly phase.

Listing 4: Gmsh physical name mapping logic

```
// Automatic mapping from Gmsh names to internal region IDs
std::string name_lower = pair.first;
std::transform(name_lower.begin(), name_lower.end(),
               name_lower.begin(), ::tolower);

if (name_lower.find("outer") != std::string::npos ||
    name_lower.find("left") != std::string::npos) {
    name_to_internal_id[pair.first] = 0;  // Outer/Left wall -> ID 0
}
else if (name_lower.find("top") != std::string::npos) {
    name_to_internal_id[pair.first] = 3;  // Top surface -> ID 3
}
```

BC Flag System: The class parses a flag vector (e.g., `bcflag = [1,1,0,0]`) from the input file, where 0 = Dirichlet, 1 = Neumann. During construction, the class classifies regions into `M_DiriRG` and `M_NeumRG`. This pre-classification allows the solvers to efficiently iterate only over relevant boundaries during system matrix assembly.

6.2.3 FEM Class

The `FEM` class serves as a wrapper around the `getfem::mesh_fem` object, abstracting the complexity of finite element space initialization. Its primary responsibilities include:

- **Data-Driven Construction:** The class constructor accepts a `GetPot` object and a section string (e.g., "Velocity", "Pressure"). It reads the specific finite element type (e.g. `FEM_PK(3,1)` or `FEM_RT0(3)`) directly from the input file, allowing runtime switching between different discretization orders and type without recompilation (for a complete list of all the possible FEM types, visit the GetFEM manual [17]).
- **DOF Management:** It provides utility methods to access the degrees of freedom (DOFs) associated with the mesh. The `getDOFpoints()` method extracts the geometric coordinates of all DOFs, which is essential for mapping spatially varying coefficients (like permeability tensors) to the discrete model.
- **Vectorial Support:** It handles the `qdim` (quotient dimension) parameter, correctly configuring the finite element space for scalar fields (Pressure, $qdim = 1$) or vector fields (Displacement/Velocity, $qdim = 3$).

6.2.4 LinearSystem Class

The `LinearSystem` class encapsulates the linear algebra backend, providing a unified interface for matrix assembly and solving. It relies on the `GMM++` library for sparse matrix storage (`gmm::row_matrix`).

- **Block-Matrix Assembly:** To support the monolithic coupled formulation, the class provides the `addSubMatrix` and `addSubSystem` methods. These allow inserting local stiffness matrices (e.g., from Darcy or Elasticity) into specific offsets (i, j) of the global system matrix, handling clearly the necessary index shifting.
- **Solver Configuration:** The solver strategy is not hardcoded, but is configured at runtime via the input file using the `configureSolver` method. We have implemented the following linear solver type:
  - **Direct Solver (SuperLU):** Configured as the default for our test cases. It performs a full LU factorization [12], which is memory-intensive but extremely robust for the indefinite saddle-point systems typical of mixed poroelasticity.
  - **Iterative Solvers:** For larger problems, the code supports GMRES and BiCGSTAB. However, these require careful preconditioning (ILU or ILUT) to handle the poor conditioning number of the coupled system.
  - **Parallel Direct Solver (MUMPS)** When memory and computational demands exceed the capacity of a single core, the code provides an interface to the MUltifrontal Massively Parallel sparse direct Solver (MUMPS) [1]. MUMPS performs a distributed LU or LDL^T factorization, efficiently exploiting multiple cores or compute nodes. This makes it suitable for problems with hundreds of thousands to millions of degrees of freedom, where the serial SuperLU factorization becomes prohibitive.

a more detailed performance analysis about the linear solvers will be presented later in section 8.4

- **Preconditioning:** It wraps GMM++ preconditioners, supporting `Identity`, `Diagonal`, `ILU`, and `ILUT` to accelerate convergence rates.

Listing 5: Solver configuration

```
// Solver types available
enum class SolverType { SUPERLU, GMRES, BICGSTAB };
enum class PreconditionerType { NONE, DIAGONAL, ILU, ILUT };
```

6.2.5 DarcyProblemT Class

This class orchestrates the solution of the time-dependent Darcy equations using a Mixed Finite Element formulation.

- **Space Setup:** It initializes two distinct `FEM` objects: `M_VelocityFEM` (typically RT_0 for flux conservation) and `M_PressureFEM` (typically P_0 for stability).
- **Matrix Assembly:** The `assembleMatrix` method builds the saddle-point system. It calls specialized operator functions such as:
  - `massHdiv`: Assembles the velocity mass matrix weighted by the inverse permeability tensor $\mathbf{K}^{-1}$.
  - `divHdiv`: Assembles the divergence constraint matrix B.
  - `massL2Standard`: Assembles the compressibility/storage term for the pressure equation.
- **Boundary Conditions:** It handles the imposition of boundary conditions using Nitsche's method for Dirichlet boundaries (weak enforcement via `essentialWNitsche`) and standard Neumann conditions.

6.2.6 ElastProblem Class

The `ElastProblem` class manages the quasi-static linear elasticity sub-problem.

- **Discretization:** It utilizes a vector-valued finite element space (`M_DispFEM`) defined on the same mesh as the Darcy problem.
- **Stiffness Assembly:** The core mechanical behavior is assembled via the `stiffElast` function, which integrates the stress tensor $\sigma(\mathbf{u}) = \lambda \text{tr}(\varepsilon)\mathbf{I} + 2\mu\varepsilon$ using the Lamé parameters provided by the data layer.
- **Boundary Enforcement:** The class supports a dual approach for Dirichlet conditions:
  - **Strong Enforcement:** Implemented via `enforceStrongBC`, which modifies the matrix rows and columns to strictly prescribe displacement values (1 on diagonal, 0 elsewhere).
  - **Weak Enforcement:** Nitsche's method implementation is available for cases requiring symmetric enforcement without matrix structure modification.

6.2.7 CoupledProblem Class

This class implements the monolithic coupling strategy, acting as the manager for the full poroelastic system.

- **Composition:** It follows the composition pattern, owning pointers to `ElastProblem` and `DarcyProblemT`. This design decouples the physics implementation from the coupling logic.
- **Coupling Assembly:** The critical interaction term, $\alpha\nabla\cdot\mathbf{u}$, is computed in `matrixFluidP`. This operator integrates the pressure shape functions against the divergence of the displacement test functions.
- **Global Assembly:** The `assembleMatrix` method aggregates the sub-matrices. It constructs the global system by placing the Elasticity stiffness matrix in the top-left block, the Darcy saddle-point system in the bottom-right, and the coupling matrices in the off-diagonal blocks.
- **Time Integration and VTK export:** It manages the time loop, updating the solution vectors for both sub-problems at each time step and exporting the results to VTK format for visualization.

Listing 6: CoupledProblem structure

```
class CoupledProblem {
protected:
    Bulk* M_Bulk;               // Domain pointer
    TimeLoop* M_time;           // Time manager
    LinearSystem* M_Sys;        // Global system
    ElastProblem* M_ElastPB;    // Elasticity sub-problem
    DarcyProblemT* M_DarcyPB;  // Darcy sub-problem
    sparseMatrixPtr_Type M_PressureStress;  // Coupling matrix
    // ...
};
```

6.2.8 PLCProblem Class

The `PLCProblem` class implements the micro-scale lacuno-canalicular (PLC) problem within the dual-porosity framework, inheriting from `CoupledProblem`.

- **Semi-Implicit Leakage Implementation:** The leakage flux $Q = \gamma(p_v - p_l)$ is handled semi-implicitly to ensure numerical stability.
  - The term dependent on the unknown pressure ($-\gamma p_l$) is moved to the Left-Hand Side and assembled into the system matrix via `LinearSystem::addToMatrix`, effectively augmenting the pressure mass matrix diagonal.
  - The source term dependent on the macro-pressure (γp_v) is added to the Right-Hand Side vector via `naturalRHS`.

- **Boundary Condition Management:** The class stores polynomial coefficients for pressure and displacement, allowing dynamic boundary condition evaluation via lambda callbacks. The methods `setOuterWallPressureBC` and `setOuterWallDisplacementBC` register these lambda callbacks that evaluate the polynomials $P(z)$ and $\mathbf{U}(z)$ (computed by the InterpolationManager) at the integration nodes of the PLC boundary.
- **Region-Based Assembly:** It supports selective assembly on specific mesh regions via the `M_outerWallRegion` member, boundary conditions are applied through lambdas only where required by the physical model, that in our case is on the outer wall of the osteon mesh.

Listing 7: PLCProblem coupling methods

```
class PLCProblem : public CoupledProblem {
public:
    void setOuterWallPressureBC(const std::vector<scalar_type>& coeffs,
                                scalar_type z_min, scalar_type z_max);
    void setOuterWallDisplacementBC(const std::vector<scalar_type>&
       coeffs_x,
                                    const std::vector<scalar_type>&
                                       coeffs_y,
                                    const std::vector<scalar_type>&
                                       coeffs_z,
                                    scalar_type z_min, scalar_type z_max
                                       );
    void setLeakageCallback(std::function<scalar_type(scalar_type)>
       pv_pressure);
    // ...
};
```

6.2.9 RussianDollProblem Class

The `RussianDollProblem` class orchestrates the complete dual-porosity coupling between the PV (vascular) and PLC (lacuno-canalicular) problems.

- **Problem Management:** It maintains pointers to both the PV (`CoupledProblem`) and PLC (`PLCProblem`) problems, along with a unique pointer to an `InterpolationManager`. This composition allows it to coordinate the sequential solution without owning the meshes directly.
- **Coupling Workflow:** The `interpolatePVtoPLC` method implements the core coupling algorithm: extracting PV solutions, performing polynomial fits via OLS, updating PLC boundary conditions, and applying the leakage term. This method is called at each time step after the PV solution is obtained.
- **Coefficient Storage:** The class stores polynomial coefficients for pressure and all three displacement components in vectors, allowing evaluation of boundary conditions at arbitrary z-coordinates. The coefficients are updated each time step based on the current PV solution.

Listing 8: RussianDollProblem workflow

```
void RussianDollProblem::interpolatePVtoPLC() {
    // 1. Get PV solutions
    auto pvPressure = M_pvProblem->getPressure();
    auto pvDisplacement = M_pvProblem->getDisplacement();

    // 2. Interpolate and fit polynomials
    auto pressureFit = M_interpManager->interpolatePressure(...);
    M_interpManager->interpolateDisplacement(..., fitX, fitY, fitZ);

    // 3. Update PLC BCs
    M_plcProblem->setOuterWallPressureBC(...);
    M_plcProblem->setOuterWallDisplacementBC(...);

    // 4. Solve PLC problem (managed externally)
}
```

6.2.10 InterpolationManager Class

The `InterpolationManager` class handles data transfer between the PV and PLC meshes, supporting two distinct approaches:

- **Profile Extraction and OLS Fitting:** This approach extracts solution values along a predefined 1D line (the osteon axis, aligned with z) from the 3D PV mesh. Instead of simple interpolation, it performs an Ordinary Least Squares (OLS) regression to reconstruct smooth polynomial profiles $P(z)$ and $\mathbf{U}(z)$ of degree N (default $N = 5$). The fitting coefficients $\mathbf{c}$ are obtained by solving the normal equations constructed via the Vandermonde matrix $\mathbf{V}$:

$$(\mathbf{V}^T\mathbf{V})\mathbf{c} = \mathbf{V}^T\mathbf{y}_{sample} \tag{23}$$

  using GMM++ dense matrix operations. This method is computationally efficient and acts as a regularization filter, removing local numerical noise from the macro-solution before applying it as boundary conditions to the PLC problem.

- **Mesh Interpolation (3D to 3D):** Alternatively, the class supports direct interpolation from the 3D PV mesh to the 3D PLC mesh using `getfem::interpolation`. However, this approach was not tested in our simulations. Due to the very small size of the osteon (the PLC domain), the number of degrees of freedom in the PLC mesh can become very large, making this approach potentially infeasible. Therefore, the line interpolation method is the preferred and implemented approach for the Russian Doll coupling.

Multi-Field Support: The class manages polynomial fits for all relevant fields: pressure (p_v) and three displacement components (u_x, u_y, u_z). The `FieldPolynomials` structure organizes these fits with a common z-range for consistent evaluation.

Callback Generation: The `createBCCallback` method generates lambda functions that evaluate the fitted polynomials at given z-coordinates, enabling dynamic boundary condition application in the PLC problem.

Listing 9: InterpolationManager data structures

```
struct PolynomialFit {
    std::vector<scalar_type> coefficients;
    scalar_type r_squared;
    scalar_type evaluateAtZ(scalar_type z) const;
};

struct FieldPolynomials {
    PolynomialFit pressure;
    PolynomialFit displacement_x, displacement_y, displacement_z;
    scalar_type z_min, z_max;
    void evaluateAll(scalar_type z, scalar_type& p,
                     scalar_type& ux, scalar_type& uy, scalar_type& uz);
};
```

6.3 Input File Format

To avoid hardcoding parameters and recompiling the code for every simulation setup, the application is fully data-driven. We use the `GetPot` library to parse a configuration file that organizes parameters into hierarchical sections.
As shown in the `data_real_parameters.txt` file included in the repository, the structure allows for fine-grained control over both physics and numerics:

- **[time]:** Defines the simulation window (`Tend`) and the time step size (`dt`).
- **[materials]:** Sets global constants such as gravity vectors and densities (ρ_r, ρ_f).
- **[bulkData/domain]:** Configures the mesh import and the strategy for boundary identification. A critical parameter here is `boundaryType`, which dictates how the solver interprets the boundary regions defined in the Gmsh file. Supported options include:
  - `tagName`: Uses physical string labels (e.g., "Outer", "Top") to map boundaries.
  - `tagNumber`: Uses the raw integer IDs from the mesh file. This is paired with `tagSelection` (values: `smallest` or `largest`) to determine the sorting order of the tags when mapping them to the internal BC flags.
  - `geometric`: Ignores mesh tags and uses automatic geometric detection (e.g., normal analysis for cylinders) to identify surfaces.
- **[bulkData/darcy] & [bulkData/mecc]:** These sections define the physical coefficients, boundary conditions and finite elements types for each subproblem.

  For coupled problems (Russian Doll), the bulk data is further divided into two distinct subsections: **[bulkDataPV]** and **[bulkDataPLC]**. This separation enables independent configuration of the PV and PLC problems, accounting for differences in porosity and elasticity properties between the two scales. This approach provides greater flexibility in defining material properties and boundary conditions for each sub-problem.

A key feature is the ability to define boundary flags as arrays (e.g., `bcflag = [1,1,0,0]`) and time-dependent loads as strings, which are interpreted at runtime. A complete input list is done in B

Listing 10: Extract from data_real_parameters.txt

```
[time]
    Tend = 7
    dt = 0.1
[../]

[bulkData]
    [./domain]
        meshExternal = bonePV.msh
        spaceDimension = 3

        % Boundary identification strategy
        boundaryType = tagNumber   % Options: tagName, tagNumber, geometric
        tagSelection = largest     % Used if boundaryType = tagNumber
    [../]

    [./darcy]
        % 0=Dirichlet, 1=Neumann
        bcflag = [1,1,0,0]
        Kxx = 10e-13
        FEMTypeVelocity = FEM_RT0(3)
        FEMTypePressure = FEM_PK(3,0)
    [../]

    [./mecc]
        % Example of parsed expression for load
        bdLoad = [0,0,1000*(x-1)*sin(t)]
    [../]
[../]
```

6.4 Expression Parser

To support the complex loading conditions seen in `data_real_parameters.txt` (e.g., `1000*(x-1)*sin(t)`), the code utilizes the `Parser` class. This component relies on a recursive descent parser to evaluate mathematical string expressions at runtime. The parser is integrated into the `BC` and `BulkData` classes, where it binds the variables x, y, z to the quadrature nodes and t to the current simulation time. This allows for the definition of spatially heterogeneous material properties or pulsating physiological loads without recompiling the C++ source code.

6.5 Assembly Operators

The assembly of finite element matrices relies on GetFEM's ***Generic Weak Form Language*** (GWFL). This high-level language allows the weak form of the equations to be expressed in tensor notation as strings, which are then compiled and optimized by the library. This approach significantly reduces the complexity of implementing mixed finite element formulations and coupled physics.

For example, the stiffness matrix for linear elasticity is assembled in `ElastOperatorsBulk.cc` using the following command:

Listing 11: Elastic stiffness matrix assembly using GetFEM GWFL

```
// Elastic stiffness matrix assembly
assem.set("lambda=data$1(#2);␣mu=data$2(#2);"
          "t=comp(vGrad(#1).vGrad(#1).Base(#2));"
          "M(#1,#1)+=␣sym(t(:,i,j,:,i,j,k).mu(k)"
          "+␣t(:,j,i,:,i,j,k).mu(k)"
          "+␣t(:,i,i,:,j,j,k).lambda(k))");
```

Here, `vGrad(#1)` represents the gradient of the displacement test functions, and the Einstein summation convention is applied to the indices (i, j, k). Similar strings are defined for the Darcy mass matrix and the poroelastic coupling terms.
For a more detailed explanation on the language, visit the GetFEM manual [17].

6.6 Parallel Implementation

We started by implementing a two-level parallelization strategy designed to accelerate both the finite element assembly phase and the linear algebra solution phase for large-scale simulations. However, we found that the assembly phase parallelization was worsening the performance of our code, so we removed it, to keep only a parallel solver.

6.6.1 Thread-Level Parallelism with OpenMP

The initial implementation included OpenMP directives to parallelize selected for-loops in the assembly routines, targeting operations such as computing nodal values for external loads or initial conditions. The implementation followed a cautious strategy: parallelization was only applied to loops performing independent assignments without race conditions. For example:

```
#ifdef _OPENMP
#pragma omp parallel for schedule(static)
#endif
for (size_type i=0; i<femC.nb_dof(); ++i) {
    datax[i] = medium->getElastData()->bulkLoad(
        femC.point_of_basic_dof(i), time)[0];
}
```

However, scalability testing revealed that this approach was *counterproductive* for our use case. As detailed in Section 8.4, increasing the number of OpenMP threads consistently *increased* execution time due to:

- Thread creation and synchronization overhead exceeding computational benefit for small loop bodies
- False sharing when multiple threads write to adjacent array elements
- Nested parallelism conflicts with GetFEM's internal OpenMP parallelization

Based on these findings, we removed all inclusion of OPENMP and loop parallelization pragmas, but it could be a possible addition to the code for future additions, especially when dealing with larger computational burdens.

6.6.2 Distributed Solver Parallelism with MUMPS

The second level of parallelism is implemented at the linear solver level using the MUltifrontal Massively Parallel sparse direct Solver (MUMPS) [1]. Unlike the serial SuperLU solver, MUMPS performs distributed LU factorization across multiple MPI processes, enabling the solution of very large linear systems (hundreds of thousands to millions of degrees of freedom) that would exceed the memory capacity of a single node. The solver is integrated via PETSc and can be selected at runtime through the input file.

It is important to note that GetFEM itself provides a parallel interface (PARA_LEVEL) that implements mesh partitioning and distributed assembly using MPI. However, this approach would require a complete redesign of the assembly routines and data structures. To avoid overcomplicating the code while still achieving significant parallel speedup for large problems, we opted for a hybrid approach: using MUMPS for distributed-memory solution parallelization, without implementing full MPI-based domain decomposition.

6.6.3 Installation and Configuration for Parallel Execution

To enable the parallel capabilities, GetFEM must be compiled with specific configuration options that differ from the standard serial installation. The key differences are:

- **Parallel Version:** Requires MUMPS for parallel linear algebra and OpenMP. The GetFEM configuration must include `-enable-openmp` and `-enable-mumps` flags.
- **Serial Version:** Uses standard BLAS/LAPACK, SuperLU, and Qhull libraries without MPI dependencies.

The `-disable-paralevel` flag in the GetFEM configuration is crucial, as it prevents the activation of GetFEM's internal MPI parallelization layer, which would conflict with our approach. This configuration allows us to leverage parallel computation in the linear solver while maintaining a simpler code structure.
Detailed step-by-step installation procedures for both serial and parallel builds are provided in Appendix C (C.3). The parallel build instructions include the exact configuration options listed above.

7 Code Performance

7.1 Compilation and Build System

We developed a flexible `Makefile` system that simplifies building the code. It automatically finds required libraries and supports both serial and parallel configurations, so users can focus on running simulations rather than setup details.

7.1.1 Architecture and Features

The Makefile implements several advanced features:

- **Intelligent Library Detection:** The build system implements a multi-level detection strategy:
  - **GetFEM:** Auto-detects based on parallel mode, with fallback to `/usr/local`, `/usr`, or the first GetFEM installation in `$HOME/getfem`. The user can also override the GETFEM_PREFIX variable to its custom path to GetFEM.
  - **BLAS/LAPACK:** First attempts `pkg-config`, then checks for OpenBLAS, finally falls back to standard `-lblas -llapack`
  - **Specialized Libraries:** SCOTCH, METIS, SuperLU, and Qhull are linked explicitly with ordering-critical dependencies
- **Flexible Main File Detection:** Supports both `main_coupled.cc` (standard Biot) and `main_russian_doll.cc` (dual-porosity) in root or `src/` directories.
- **Automatic Dependency Tracking:** Generates `.d` files via `-MM -MT` flags, ensuring incremental rebuilds only recompile changed sources and their dependents.
- **Warning Suppression:** Systematically filters third-party library warnings, in order to have a clean compilation output, with the only exception of the `BOOST_HEADER_DEPRECATED` pragmas, which couldn't be suppressed, but are harmless. We used the following suppression:
  `-Wno-comment`,
  `-Wno-unused-but-set-variable`,
  `-Wno-unused-parameter`,
  `-Wno-pragmas`,
  `-Wno-unknown-pragmas`,
  `-Wno-deprecated-declarations`,
  `-Wno-misleading-indentation`,
  `-Wno-cast-function-type`.

7.1.2 Build Configurations

The unified Makefile supports both serial and parallel execution modes through a single, intelligent configuration system. Three build optimization levels are available via the `BUILD_TYPE` variable:

Mode	Flags	Purpose
`release`	`-O3 -DNDEBUG -march=native`	Production builds (default); aggressive optimization including loop unrolling, function inlining, and CPU-specific instructions
`debug`	`-g -O0 -DDEBUG`	Development builds with full debug symbols and no optimization
`profile`	`-g -O2 -pg`	Performance profiling with `gprof`

Parallel mode can be invoked in two equivalent ways:

- **As a target:** `make parallel russian` (recommended)
- **As a variable:** `make PARALLEL=1 russian`

The Makefile automatically detects the `parallel` target and sets `PARALLEL=1`.
When `PARALLEL=1` is set, the build system automatically:

1. Selects the parallel GetFEM installation (`getfem-5.4.4-parallel`)
2. Defines `-DUSE_MUMPS` to enable MUMPS solver code paths
3. Links all required parallel libraries (OpenMP, MUMPS with its dependencies, and MPI) in the correct order
4. Ensures thread-safe execution with proper runtime configuration

A `VERBOSE_MODE` flag enables `#ifdef VERBOSE` sections for detailed runtime diagnostics.

7.1.3 Usage Examples

The Makefile provides two equivalent syntaxes for parallel builds:

```
# Serial builds (default)
make russian -j4
make coupled

# Parallel builds - Method 1 (recommended, concise)
make parallel russian -j4
make parallel coupled

# Parallel builds - Method 2 (explicit flag)
make PARALLEL=1 russian -j4
make PARALLEL=1 coupled

# Debug builds
make BUILD_TYPE=debug russian               # Serial debug
make parallel BUILD_TYPE=debug russian      # Parallel debug

# Custom GetFEM location
make GETFEM_PREFIX=/custom/path/getfem-parallel russian

# Verbose diagnostics
make VERBOSE_MODE=1 coupled
make parallel VERBOSE_MODE=1 russian
```

7.1.4 Execution

Serial and parallel builds require different runtime configurations:

```
# Serial execution
./main_russian_doll -f input/data_russian.txt

# Parallel execution (single process, multi-threaded)
export OPENBLAS_NUM_THREADS=4
./main_russian_doll -f input/data_russian.txt
```

```
# NOT: mpirun -np 4 ./main_russian_doll
# (MUMPS uses internal threading, not MPI process parallelism)
```

Critical distinction: The parallel mode uses single-process, multi-threaded parallelism through MUMPS with MPI as a supporting library. It does *not* employ distributed MPI parallelism (GetFEM `PARA_LEVEL=2`), avoiding the complexity of domain decomposition while achieving a noticeable speedup for medium to large sized problems. The `-disable-paralevel` GetFEM configuration flag ensures no distributed parallelism is attempted.

7.1.5 Build System Diagnostics

The Makefile provides comprehensive diagnostic targets:

```
make info              # Display complete build configuration
make check-getfem      # Verify GetFEM installation and mode
make list-mains        # Show detected main source files
make help              # Complete usage documentation
```

All generated files (objects, executables) are removed via `make clean`, with `make distclean` additionally removing temporary files (`*.vtk`, `*.mm`, `output_vtk/`).

7.2 Memory Management

Given the 3D nature of the problem, memory efficiency is a primary concern.

Sparse Storage. We utilize the `gmm::row_matrix<gmm::rsvector<scalar_type>>` format. This Compressed Sparse Row (CSR) structure stores only non-zero entries, which is essential since the connectivity of tetrahedral meshes results in matrices with sparsity patterns $> 99\%$.

Reuse Policy. Since we assume small deformations (linear poroelasticity) and constant permeability, the stiffness (K_{uu}), mass (M_{pp}), and divergence (B) matrices are assembled only once at the beginning of the simulation (see Table 2). Only the Right-Hand Side (RHS) vectors are recomputed at each time step, significantly reducing the computational cost per iteration.

Table 2: Matrix computation and reuse strategy

Matrix	**Computed**	**Reused for**
Elasticity stiffness A_{uu}	Once (linear)	All time steps
Darcy velocity mass A_{qq}	Once	All time steps
Darcy divergence B	Once	All time steps
Pressure mass M_{pp}	Once	Time-stepping, errors
Poroelastic coupling A_{up}	Once	All time steps

8 Profiling and Bottlenecks

An analysis of the code execution reveals where the computational resources are concentrated as follows:

- **Linear Solve ($\sim 60 - 70\%$):** The majority of wall-clock time is spent inside the `solve()` method of `LinearSystem`. For large problems with direct solvers like SuperLU, this percentage increases significantly. This confirms that future optimization efforts should focus on efficient solver selection and, where applicable, parallel algebraic solvers.
- **Assembly ($\sim 20-30\%$):** The assembly phase, while benefiting from the "assemble-once" strategy in `CoupledProblem`, still represents a significant portion of runtime for the initial matrix construction. The overhead of the GetFEM generic assembly language provides flexibility at the cost of some performance compared to hand-coded element routines.
- **I/O Operations ($\sim 5-10\%$):** The export of VTK files via `UsefulFunctions::exportSolution` and CSV history logging can become noticeable for large meshes. For production runs, exporting every N-th frame is recommended to avoid I/O bottlenecks.

Computational Environment

All simulations were conducted on a system with the following specifications:

- **CPU**: Intel Core i7-11th Gen, 2.8 GHz, 4 cores, 8 threads
- **RAM**: 16 GB DDR4
- **Storage**: 512 GB NVMe SSD
- **Operating System**: Ubuntu 22.04 LTS
- **Compilation**: GCC 11.3.0 with optimization level -O3

8.1 Memory Analysis

Memory profiling was conducted using Valgrind's memcheck tool to identify potential memory leaks. The tests were performed using the `data_2d_square_convergence` data file, with a discretization of 100x100.

8.1.1 Leak Detection Results

Valgrind analysis revealed minimal memory leaks in the codebase:

Table 3: Valgrind memory leak summary

Category	Size (bytes)	Assessment
Definitely lost	32	Minor, one-time allocation
Indirectly lost	0	None
Possibly lost	2,352	Thread-local storage (normal)
Still reachable	329,793	Global tables (intentional)

8.1.2 Analysis of Detected Leaks

32-byte Definite Leak. This minor leak originates in the `LifeV::Parser::setString()` method, specifically within the Boost.Spirit grammar initialization. The leak occurs once during `BulkDarcyData` construction and does not grow with problem size or number of time steps. Given that typical simulations use 500 MB–1 GB of memory, this 32-byte leak represents approximately 0.000003% of total memory usage and is negligible for practical purposes.

Thread-Local Storage (2,352 bytes). The "possibly lost" memory is allocated by the OpenMP runtime for thread-local storage when worker threads are created. This is normal behavior and the memory is properly freed when threads are destroyed at program termination. Valgrind cannot always track this cleanup correctly.

Still Reachable Memory (329 KB). The largest category consists of:

- GetFEM's global polynomial coefficient tables (∼180 KB from `bgeot_poly.cc`)
- Boost.Spirit parser caches
- Static initialization data

This memory is intentionally retained for performance reasons—these tables are expensive to compute and are reused throughout the simulation. This is a design choice, not a leak.

8.1.3 Memory Leak Conclusions

The memory analysis confirms that the implementation is essentially leak-free. The only true leak (32 bytes) is inconsequential and does not affect scalability. No corrective action is required, though future work could optimize the Parser class to avoid repeated `setString()` calls during assembly, which would both eliminate the minor leak and improve performance.

8.2 Methodology

Three mesh resolutions were used for scalability analysis:

1. **Small**: 10×10 mesh (Darcy DOFs: 520; Elastic DOFs: 242; Total $\approx$ 762)

2. **Medium**: 100×100 mesh (Darcy DOFs: 50,200; Elastic DOFs: 20,402; Total $\approx$ 70,602)

3. **Large**: 500×500 mesh (Darcy DOFs: 1,251,000; Elastic DOFs: 502,002; Total $\approx$ 1,753,002)

The time domain was discretized with $T = 10$ and $\Delta t = 0.2$, resulting in 50 time steps. Iterative solvers were configured with tolerance 10^{-7} and maximum 10,000 iterations.

8.3 Performance Monitoring Method

A custom `PerformanceMonitor` class (Listing 12) was implemented to track:

- Total runtime
- Time per time step
- Individual solver times
- Assembly and RHS computation times
- Memory usage (indirectly via system monitoring)

using the `std::chrono::high_resolution_clock` as main time measuring class. To maintain a clean codebase, the `PerformanceMonitor` class was used strictly for profiling phases and excluded from the final production build.

Listing 12: PerformanceMonitor class (abridged)

```
class PerformanceMonitor {
public:
    struct TimingData {
        double total_time = 0.0;
        double min_time = std::numeric_limits<double>::max();
        double max_time = 0.0;
        size_t call_count = 0;
    };

    static PerformanceMonitor& instance();
    void startTimer(const std::string& name);
    double stopTimer(const std::string& name);
    void printSummary() const;
    void exportToCSV(const std::string& filename) const;
};
```

8.4 Results

Solver Performance Comparison. Table 4 summarizes the performance of all four solvers across different problem sizes.

Key observations:

- For small problems (10×10), all solvers perform comparably, with BiCGSTAB slightly faster due to lower setup overhead.

Table 4: Solver performance for different mesh sizes. Times in seconds.

2*Solver	10×10 Mesh		100×100 Mesh		500×500 Mesh	
	Total	Time/Step	Total	Time/Step	Total	Time/Step
SuperLU	2.202	0.04	137.26	2.75	N/A (OOM)	N/A
MUMPS	3.603	0.06	161.53	3.10	4205.15	84.10
GMRES	2.175	0.04	582.50	11.50	Partial	1530
BiCGSTAB	2.010	0.038	231.99	4.41	25011.42	500.23

- Direct solvers (SuperLU, MUMPS) show their advantage at medium scale through robust convergence.
- For large problems, MUMPS is the clear winner among direct solvers, while BiCGSTAB provides a viable iterative alternative when memory is constrained.

Convergence Behavior. Figure 1 shows the convergence history for iterative solvers. GMRES achieves better residual reduction but at higher computational cost per iteration.

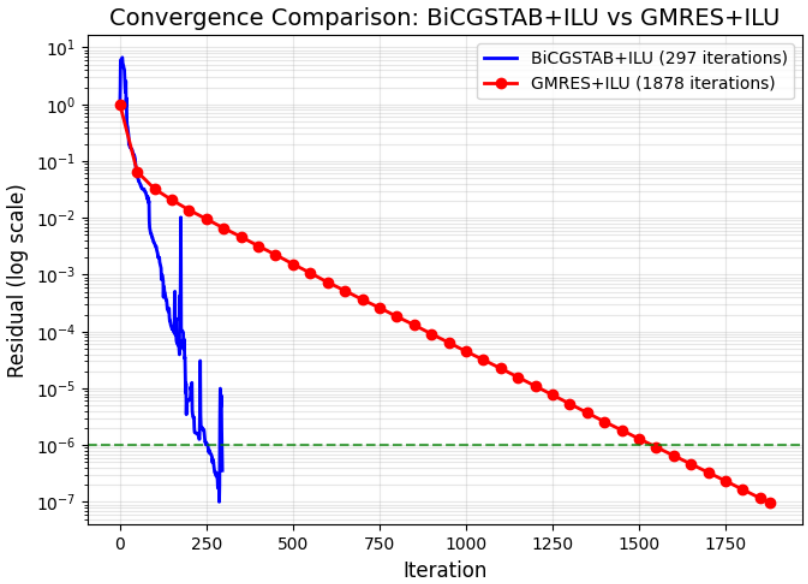


Figure 1: Convergence history of iterative solvers (medium mesh)

Parallel Assembly Performance. The assembly phase was analyzed for OpenMP parallelization potential. However, testing revealed that the manual OpenMP pragmas applied to data preparation loops actually *decreased* performance due to:

- Thread creation and synchronization overhead exceeding the benefit for small loop bodies
- False sharing when multiple threads write to adjacent array elements
- Nested parallelism conflicts with GetFEM's internal OpenMP parallelization

Table 5 shows the observed behavior when varying `OMP_NUM_THREADS`:

Table 5: Effect of OpenMP thread count on initialization time (500×500 mesh)

Threads	Init Time (s)	Relative to 1 Thread
1	51.02	1.00×
2	55.27	0.92×
4	59.88	0.85×
8	65.84	0.77×

This *inverse scaling* indicates that the manual OpenMP parallelization is counterproductive. The recommended approach is to remove the manual `#pragma omp` directives from the assembly files and rely on GetFEM's internal parallelization, or to ensure that only sufficiently large, compute-bound loops are parallelized.

MUMPS Thread Scalability. While manual OpenMP parallelization of assembly proved counterproductive, the MUMPS solver provides internal parallelization through multi-threaded BLAS operations. Table 6 presents the effect of varying `OPENBLAS_NUM_THREADS` on MUMPS solver performance.

Table 6: MUMPS solver scalability with OPENBLAS thread count

100×100 Mesh (70,602 DOFs)				
Threads	Total Time (s)	Mean Solve (s)	Min Solve (s)	Max Solve (s)
1	248.37	0.79	0.63	1.19
2	239.40	0.77	0.63	1.06
4	228.70	0.75	0.635	1.00
8	238.19	0.76	0.607	1.19
500×500 Mesh (1,753,002 DOFs)				
Threads	Total Time (s)	Mean Solve (s)	Min Solve (s)	Max Solve (s)
1	1022.31	16.22	13.57	23.93
2	890.24	13.83	13.39	15.29
4	960.52	14.76	14.45	15.06
8	1139.73	18.53	13.47	26.75

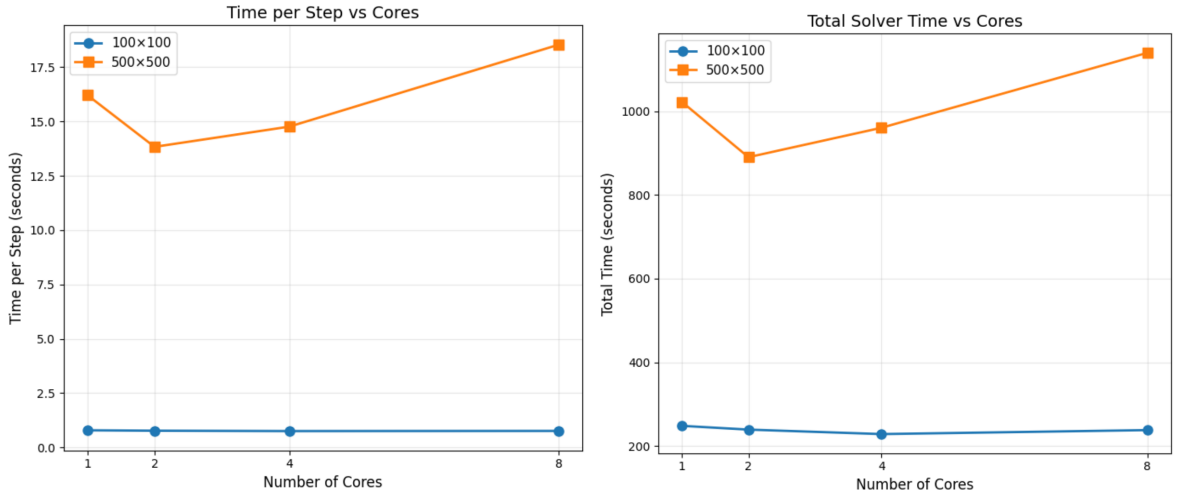
Key observations:

- For the medium mesh (100×100), modest speedup is achieved with 4 threads (228.7s vs 248.4s baseline, ∼1.09× speedup), but 8 threads shows performance degradation.
- For the large mesh (500×500), optimal performance occurs at 2 threads (890s vs 1022s baseline, ∼1.15× speedup). Both 4 and 8 threads show diminishing returns or performance degradation.

- The non-monotonic scaling behavior is characteristic of memory-bandwidth limited operations: additional threads compete for shared memory resources, and cache coherency overhead exceeds computational gains.
- The high variance in per-step timing (e.g., 13.47–26.75s for 8 threads on the large mesh) indicates thread scheduling instability at higher thread counts.

Based on these results, **2–4 OPENBLAS threads** is recommended for MUMPS on typical workstation hardware. The optimal thread count depends on the specific CPU architecture and should be determined empirically for production runs.

Figure 2 illustrates the scalability trends for the medium and large mesh cases.



(a) Time per time step

(b) Total simulation time

Figure 2: Scalability analysis for medium mesh (100×100)

8.5 Solver Performance Analysis and Recommendations

8.5.1 Solver Characteristics and Performance

Direct Solvers: SuperLU demonstrated excellent performance for small to medium problems but failed for large meshes due to excessive memory requirements ($O(N^{1.5})$ fill-in for 2D problems). MUMPS showed similar memory characteristics but successfully handled problems up to 1.7M DOFs through more efficient memory management and internal multi-threading.

Iterative Solvers: BiCGSTAB proved to be the most practical iterative solver, offering the best trade-off between convergence speed and memory efficiency. While GMRES achieved mathematically superior convergence rates, it proved impractical for production runs due to excessive memory requirements (from storing Arnoldi vectors) and wall-clock time.

Parallelization Approaches: Two strategies were evaluated:

1. **Manual OpenMP (assembly loops)**: Proved counterproductive due to overhead exceeding benefits. *Not recommended.*

2. **MUMPS internal threading (via OPENBLAS)**: Provides modest speedup (1.1–1.15×) with 2–4 threads. *Recommended for production use.*

The limited parallel efficiency is consistent with the memory-bandwidth-limited nature of sparse matrix operations. Significant parallel speedup would require distributed-memory parallelization (MPI-based domain decomposition), which is beyond the scope of the current implementation.

8.5.2 Scalability Limitations

The study revealed several key scalability limitations:

1. **Memory constraints**: Direct solvers require $O(N^{1.5})$ memory for 2D problems
2. **Condition number growth**: The coupled poroelasticity system's condition number increases with mesh refinement, degrading iterative solver convergence
3. **Parallelization overhead**: Manual OpenMP parallelization proved counterproductive for the assembly phase

8.5.3 Optimal Solver Selection Guide

Based on the comprehensive evaluation, the following recommendations emerge:

- **Small problems (<10,000 DOFs)**: Any solver performs adequately. **BiCGSTAB** offers the fastest execution due to minimal setup overhead, while **SuperLU** provides the most robust convergence.
- **Medium problems (10,000–100,000 DOFs)**: **MUMPS** is recommended for its combination of robustness and reasonable performance. If memory is constrained, **BiCGSTAB** with ILU preconditioning is a viable alternative.
- **Large problems (>100,000 DOFs)**: Two strategies are recommended:
    1. **MUMPS** (if memory permits): Provides reliable convergence and competitive performance up to approximately 2M DOFs on systems with 16+ GB RAM.
    2. **BiCGSTAB** (memory-constrained): Scales to larger problems with $O(N)$ memory complexity, though convergence may require more iterations for ill-conditioned systems.
- **Not recommended**: GMRES without advanced preconditioning, due to excessive memory and time requirements for this problem class.

8.5.4 Summary

Direct solvers (SuperLU, MUMPS) demonstrated robust convergence where memory permitted, with MUMPS emerging as the preferred choice for larger problems due to its more efficient memory management. Among iterative solvers, BiCGSTAB provided the best practical trade-off between convergence speed and memory efficiency. Parallelization efforts showed limited success, with only MUMPS internal threading providing modest speedup. The primary scaling limitations remain memory constraints for direct solvers and arithmetic intensity for iterative methods.

Table 7: Solver selection guide

Criterion	SuperLU	MUMPS	GMRES	BiCGSTAB
Small problems	✓✓	✓	✓	✓✓
Large problems	×	✓✓	×	✓✓
Memory efficiency	×	×	✓	✓✓
Robustness	✓✓	✓✓	✓	✓
Parallel scaling	×	✓	✓	✓

9 Numerical Results

9.1 Solver Validation: Convergence Analysis

To validate the correctness of the implemented coupled poroelastic solver and ensure the stability of the mixed finite element formulation, we did a convergence test using the Method of Manufactured Solutions (MMS).

Problem Setup. The test considers a 2D unit square domain $\Omega = [0, 1] \times [0, 1]$. We simulate a steady-state regime (approximated numerically by using a large time step $\Delta t = 10^4$) to verify the spatial discretization error. To rigorously test the coupling between the Darcy flow and the structural mechanics, we set the Biot coefficient $\alpha = 1.0$ and $M = 1.0$.
An exact analytical solution is imposed for the pressure field p, while the displacement field $\mathbf{u}$ is constrained to remain zero. This specific setup isolates the solver's ability to correctly balance the internal pore pressure forces with the external body forces.
The exact solution is defined as:

$$p_{\text{exact}}(x, y) = \sin(\pi x) \sin(\pi y), \quad \mathbf{u}_{\text{exact}}(x, y) = \mathbf{0} \tag{24}$$

To satisfy the governing equations, we derived the following source terms:

- **Fluid Source:** To satisfy $-\Delta p = f$, a source term $S_f = 2\pi^2 \sin(\pi x) \sin(\pi y)$ is applied.
- **Mechanical Body Load:** To maintain static equilibrium ($\mathbf{u} = \mathbf{0}$) against the pressure gradient force ($\alpha \nabla p$), a balancing body force $\mathbf{f}_{ext} = -\alpha \nabla p$ is applied:

$$\mathbf{f}_{ext} = \begin{bmatrix} -\pi \cos(\pi x) \sin(\pi y) \\ -\pi \sin(\pi x) \cos(\pi y) \end{bmatrix} \tag{25}$$

Discretization and Parameters. The domain was discretized using a structured triangular mesh and to ensure the inf-sup stability (LBB condition) required for the mixed formulation, we chose the following Finite Element spaces:

- **Displacement:** P_1 (Continuous piecewise linear).
  Theoretical convergence rate in L^2 norm: $O(h^2)$.
- **Velocity (Darcy):** RT_0 (Raviart-Thomas order 0).

- **Pressure:** P_0 (Piecewise constant).
  Theoretical convergence rate in L^2 norm: $O(h)$.

Boundary conditions were set to homogeneous Dirichlet for the pressure ($p = 0$ on $\partial\Omega$) to match the vanishing boundary values of the chosen exact solution and homogeneous Dirichlet for the displacement ($\mathbf{u} = \mathbf{0}$ on $\partial\Omega$).

Convergence Results. We ran the simulation on three progressively refined meshes ($N = 4, 8, 16$). The L^2 errors for both pressure and displacement were computed against the exact solution and the results are summarized in Table 8.

Mesh Refinement	h	Error Pressure (L^2)	Error Displacement (L^2)
$N = 4$	0.2500	1.08×10^{-2}	4.84×10^{-2}
$N = 8$	0.1250	2.93×10^{-3}	1.26×10^{-2}
$N = 16$	0.0625	7.51×10^{-4}	3.13×10^{-3}

Table 8: Convergence analysis for the coupled poroelastic problem.

The estimated order of convergence (EOC) was calculated using the logarithmic slope of the error reduction and the results demonstrate a quadratic convergence rate for both fields (see Figure 3):

- **Pressure EOC:** ≈ 1.97. Although the standard a priori estimate for P_0 elements predicts a linear convergence ($O(h)$), we observe a quadratic rate ($O(h^2)$). We attributed this *superconvergence* behavior to the regularity of the structured mesh and the symmetry of the exact solution.
- **Displacement EOC:** ≈ 2.01. This perfectly matches the theoretical expectation for P_1 elements ($O(h^2)$).

This result confirms that the coupling terms are correctly implemented. The displacement error converging to zero confirms the correct balancing of the pressure gradient by the imposed body forces.

9.2 Test Case: 2D Annulus

To demonstrate the flexibility of the implemented C++ classes, we simulated a poroelastic problem on a 2D annulus geometry. This test case is particularly significant as it validates the "Smart Default" logic implemented in the `CoupledProblem`, `DarcyProblemT` and `ElastProblem` classes, which automatically detect the mesh dimension and adapt the finite element integration methods (switching from `IM_TETRAHEDRON` to `IM_TRIANGLE`) without requiring recompilation.

Problem Setup. The mesh was generated using Gmsh and imported via the `Bulk` class.

- **Geometry:** 2D Annulus with unstructured triangular mesh ($N_{elements} = 2818$).

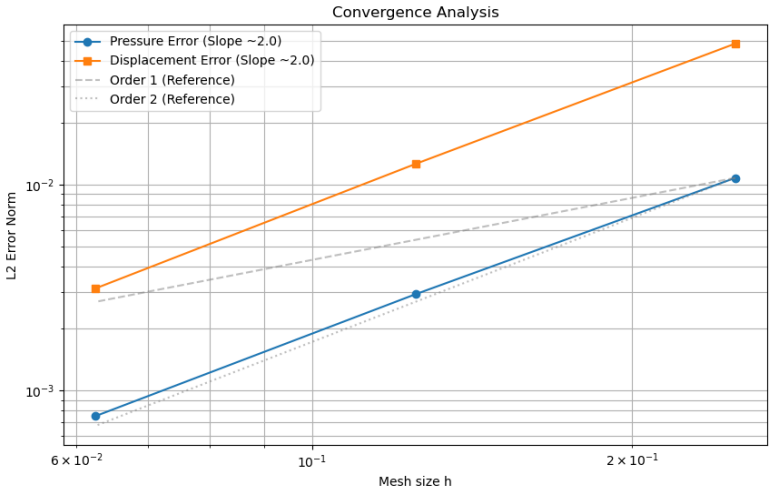


Figure 3: Log-log plot of the L^2 error convergence for Pressure and Displacement. Both fields exhibit second-order convergence ($O(h^2)$).

- **Degrees of Freedom:** The total system size is 9503 DOFs (6685 for Darcy flow, 2818 for Elasticity).
- **Boundary Conditions:** To perform a constant-pressure patch test on a curved domain, we imposed:
  - **Fluid:** Dirichlet condition $P = 10$ Pa on **both** inner and outer walls to ensure a uniform pressure field.
  - **Mechanics:** Homogeneous Dirichlet condition ($\mathbf{u} = \mathbf{0}$) on all boundaries to clamp the solid skeleton.
- **Material:** Isotropic porous medium with unitary permeability and stiffness parameters ($\mu = 1, \lambda = 1$).

Results and Solver Performance. The simulation was performed over 5 time steps with $\Delta t = 0.2$. As shown in the execution logs, the code successfully mapped the Gmsh physical tags ("Inner", "Outer") to the internal boundary regions, preventing the segmentation faults typical of dimensional mismatch.

Solver Convergence: We utilized the `BiCGSTAB` iterative solver preconditioned with `ILU(0)` and the solver demonstrated excellent stability. Despite the saddle-point nature of the mixed formulation, the solver successfully converged at every time step within the tolerance of 10^{-8}.

Error Analysis and Geometric Effects: The numerical errors computed against the constant exact solution ($P_{ex} = 10, \mathbf{u}_{ex} = \mathbf{0}$) at the final time $T = 1.0$ were:

- **Pressure Error (L^2):** ≈ 0.52
- **Displacement Error (L^2):** ≈ 0.46

Unlike the unit square test case where errors converged to machine precision, the annulus test exhibits a bounded discretization error. This is physically consistent with the finite element approximation employed:

1. **Polygonal Approximation:** The curved boundaries of the annulus are approximated by straight edges in the mesh: this geometric discrepancy introduces a systematic error in the application of boundary conditions.

2. **P0 Elements Gradient:** Using piecewise constant (P_0) elements for pressure means the numerical solution is a step function. Even if the global field is constant, small local fluctuations due to the mesh irregularity create non-zero numerical gradients ($\nabla p_h \neq 0$).

3. **Coupling-Induced Displacement:** This is the critical factor. The momentum balance equation in Biot's theory contains the coupling term $-\alpha \nabla p$, which acts as an internal body force. Analytically, for a constant pressure, $\nabla p = \mathbf{0}$, so this force should vanish. Numerically, however, the spurious gradients $\nabla p_h \neq \mathbf{0}$ (described in point 2) generate artificial forces on the solid skeleton. Since the material is relatively compliant ($\mu = 1$), these numerical "phantom forces" deform the mesh, resulting in the observed non-zero displacement error (≈ 0.46).

This test confirms that the solver correctly handles complex geometries and coupled physics, while highlighting the importance of mesh refinement and higher-order elements for curved domains.

9.3 Test Case: 3D Unit Cube

To verify the full 3D capabilities and temporal stability of the solver, we simulated a poroelastic unit cube $\Omega = [0, 1]^3$ under transient conditions using a semi-structured tetrahedral mesh.

Problem Setup. We employed the Method of Manufactured Solutions (MMS) with a time-dependent exact solution characterized by high-order spatial variability and linear time evolution:

$$p_{ex}(x, y, z, t) = t \cdot x(1 - x)y(1 - y)z(1 - z) \tag{26}$$

$$\mathbf{u}_{ex}(x, y, z, t) = [p_{ex}, p_{ex}, p_{ex}]^T \tag{27}$$

The forcing terms (Darcy source and Elastic body forces) were derived analytically and applied via the input file parser.

- **Discretization:** $16 \times 16 \times 16$ spatial grid ($N_{dof} \approx 90,000$), $\Delta t = 0.1$s.
- **Duration:** $T_{end} = 1.0$s (10 time steps).
- **Solver:** BiCGSTAB with ILU preconditioner.

Results and Error Evolution. The solver demonstrated robust stability throughout the simulation. The monolithic linear system converged in approximately 70-85 iterations per time step, with residuals consistently dropping below 10^{-13}.
Table 9 reports the evolution of the L^2 error norms over time.

Table 9: Error evolution for the 3D Cube test case.

Time (s)	**Step**	$\|\|E_p\|\|_{L^2}$	$\|\|E_u\|\|_{L^2}$
0.1	1	4.75×10^{-4}	3.73×10^{-4}
0.5	5	2.84×10^{-3}	1.86×10^{-3}
1.0	10	5.88×10^{-3}	3.72×10^{-3}

Discussion on stability. As shown in Table 9, the error increases monotonically with time. This behavior is expected and physically consistent for two reasons:

1. **Temporal Accumulation:** The Implicit Euler scheme employed for time integration is first-order accurate ($O(\Delta t)$). In transient problems, the local truncation error accumulates linearly with the number of time steps.

2. **Spatial Interpolation:** The exact solution involves a 6th-degree polynomial in space, while the pressure is discretized with piecewise constant (P_0) elements. This creates a significant spatial interpolation error ($O(h)$) that dominates the absolute value of the error.

Crucially, the error growth is linear rather than exponential, confirming the **unconditional stability** of the implemented numerical scheme for 3D coupled problems.

9.4 Test Case: Realistic Bone Geometry

To validate the solver under realistic biomedical conditions, we simulated a 3D segment of an osteon modeled as a hollow cylinder. Several mechanical loading scenarios were simulated in order to investigate how fluid pressures and flows respond to different external loads.

9.4.1 Axial Compression (real parameters)

Problem Setup. The first scenario consists of applying a boundary load on the top surface of the cylinder while the bottom surface is fixed. This simulates compressive loading as experienced by long bones during standing or walking. The loading intensity depends on both the time and the x coordinate of the node on which the force is applied. The simulation setup replicates the conditions of a single osteon embedded in the cortical matrix, utilizing the parameters defined in `data_real_parameters.txt`, which are the experimentally derived values reported in [18]:

- **Geometry:** Unstructured tetrahedral mesh of a hollow cylinder.
- **Parameters:**
  - Permeability: $K = 10^{-13}\, m^2$ (isotropic).
  - Stiffness: $\mu \approx 6.0$ GPa, $\lambda \approx 9.1$ GPa (Young's Modulus $E \approx 14$ GPa).
  - Viscosity: $\mu_f = 0.001\, \text{Pa} \cdot \text{s}$ (Water-like).

- **Loading:** A time-dependent sinusoidal load is applied to the top surface to mimic cyclic mechanical loading:

$$\mathbf{t}_{load} = [0, 0, 1000 \cdot (x - 1) \cdot \sin(t)]^T \quad \text{for } z > 0.57 \tag{28}$$

  This load creates a bending moment on the osteon.

- **Solver:** The linear system was solved using the `SuperLU` direct solver, which provides robust factorization for the indefinite saddle-point matrix arising from these stiff parameters.

Results and Discussion. The results are shown in Figure 4. The simulation completed 10 time steps ($\Delta t = 0.1$s) without stability issues.

- **Displacement Field**: Under the applied load of 1 kPa, the computed L^2 norm of the displacement field was approximately 1.05×10^{-8} m at $t = 1.0$s. Given the high stiffness of the bone ($E \sim 10^{10}$ Pa), the expected strain is in the order of $\epsilon \approx \sigma/E \approx 10^{-7}$. The numerical results are therefore physically consistent with linear elasticity theory.

- **Pressure Generation**: The computed pore pressure norm was in the order of 10^{-10} Pa. This low value is expected because the applied mechanical strain rate was relatively slow compared to the fluid diffusion time scale, allowing the fluid to partially equilibrate. However, the non-zero pressure field confirms that the fluid-structure interaction is active: mechanical deformation successfully induces pressure gradients within the porous matrix, validating the coupling implementation.

Conclusion on Applicability. This test confirms that the developed software can handle real-world physical units and parameters. The stability of the monolithic solver with bone parameters suggests it is suitable for patient-specific simulations derived from micro-CT scans.

9.4.2 Local Load (ideal parameters)

This test case evaluates the transient response of the coupled system to a spatially concentrated and time-varying mechanical load. The objective is to verify the solver's ability to resolve localized deformations without generating spurious oscillations in the global pressure field.

Problem Setup. The simulation setup is derived from the ideal cylindrical configuration but introduces a highly localized Neumann boundary condition.

- **Geometry and Parameters:** We utilize the standard hollow cylinder mesh with unitary material parameters ($\mu = 1, \lambda = 1, K = 1$) to isolate the numerical response from material-specific scaling effects.

- **Boundary Constraints:** The inner cylindrical surface is fully clamped (Dirichlet $\mathbf{u} = \mathbf{0}$), while the outer surface, top, and bottom faces are traction-free (Neumann).

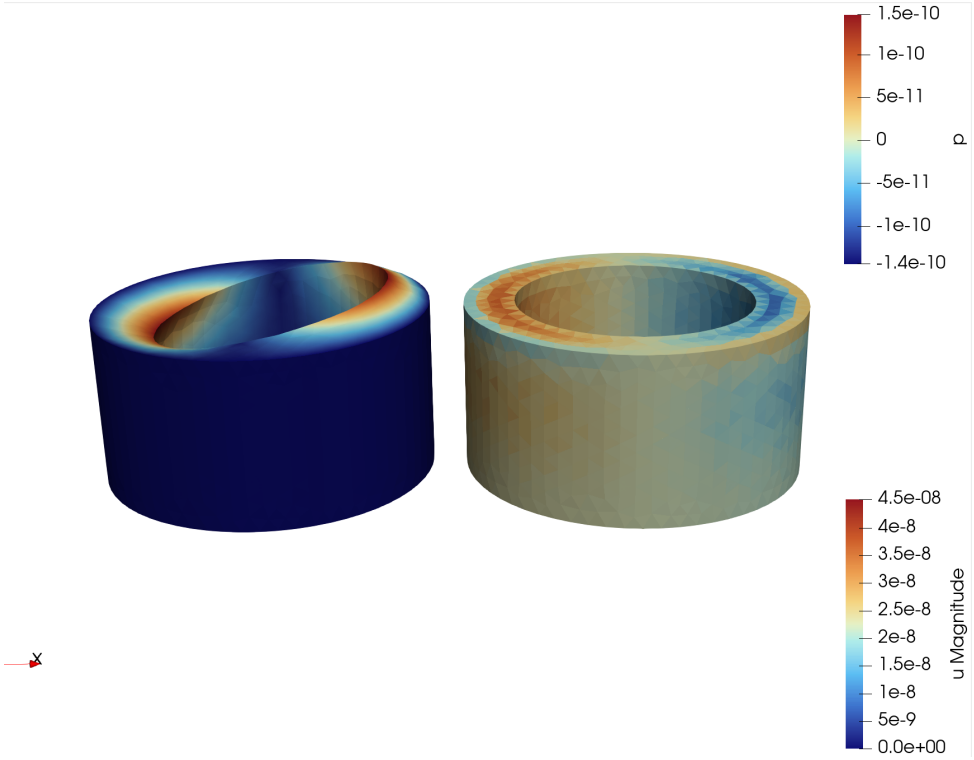


Figure 4: Numerical results for the axial compression test at final time $T = 1.0$s. **Left:** Displacement magnitude field [m]. **Right:** Pore pressure distribution [Pa].

- **Load Definition:** A vertical compressive force F_z is applied on a specific sub-region of the top surface. The load is defined by a composite function of space and time:

$$\mathbf{g}(x, y, z, t) = \begin{bmatrix} 0 \\ 0 \\ -10 \cdot \sin(t) \cdot \mathbb{I}_{\{z>0.5\}} \cdot \mathbb{I}_{\{x>1.5\}} \cdot \mathbb{I}_{\{y<0.5\}} \end{bmatrix} \tag{29}$$

  where $\mathbb{I}_{\{\cdot\}}$ is the indicator function. This creates a "piston-like" force acting only on the front-right quadrant of the top ring, oscillating sinusoidally in time.

Results. The simulation was executed using the `BiCGSTAB` iterative solver with ILU preconditioning. As shown in the convergence logs, the solver maintained stability with a relatively low number of iterations (averaging 38-42 iterations per time step) despite the discontinuous nature of the applied load.
The resulting displacement field (Figure 5) exhibits a clear localized depression in the loaded area, with the deformation magnitude fading to zero towards the clamped inner wall.

- **Mechanical Response:** The displacement peak follows the sinusoidal temporal profile of the load, reaching a maximum magnitude of $||\mathbf{u}||_{L^2} \approx 0.49$ at $t \approx \pi/2$ (peak load).

- **Coupled Response:** The induced pore pressure field is non-zero but remains smooth, confirming that the mixed finite element formulation ($\mathbb{RT}_0 - \mathbb{P}_0$) effec-

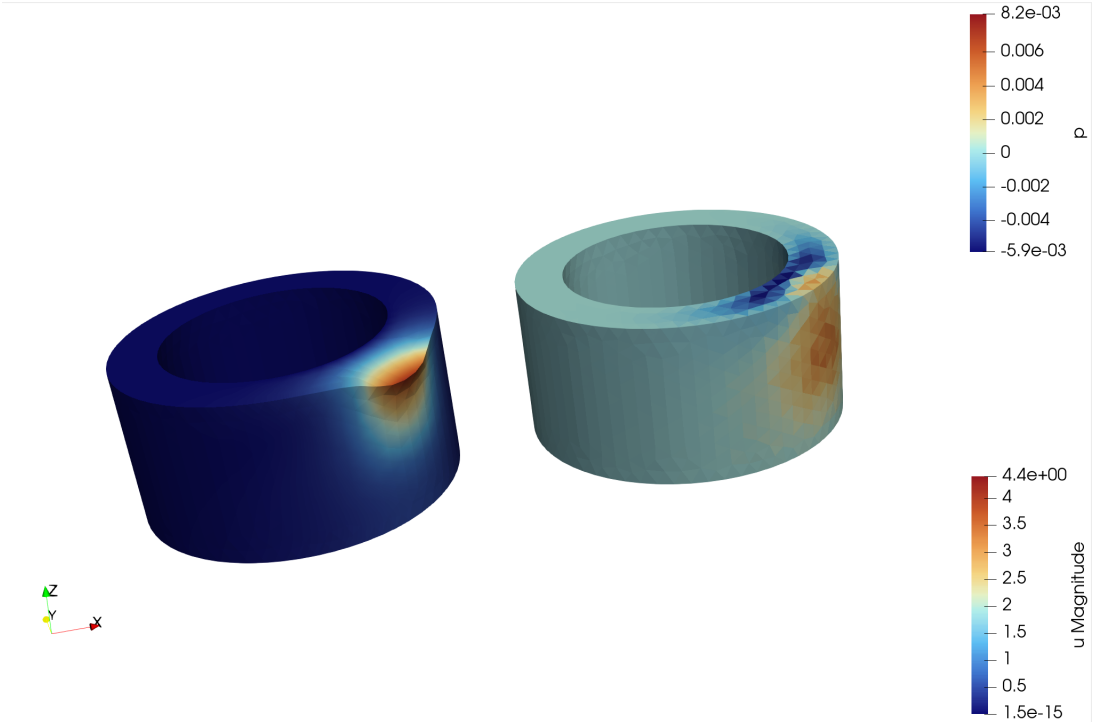


Figure 5: Numerical results for the local load test at $t = 1$s. **Left:** Displacement magnitude field [m]. **Right:** Pore pressure distribution [Pa].

tively filters out spurious pressure modes that can arise from sharp gradients in the displacement field.

9.4.3 Rotational Loading (ideal parameters)

The final test case investigates the behavior of the osteon segment under a torsional load. This scenario is clinically relevant as torsional forces are a common cause of spiral fractures in long bones [6, 14]. From a numerical point of view, this test verifies the solver's ability to handle complex, spatially-varying vector loads that induce shear stresses.

Problem Setup. The geometry remains the standard hollow cylinder. To simulate pure torsion, the boundary conditions are defined as follows:

- **Base Constraint:** The bottom surface ($z = 0$) is fully clamped ($\mathbf{u} = \mathbf{0}$) via a homogeneous Dirichlet condition.
- **Torsional Load:** A tangential traction force is applied to the top surface ($z = 1$). The force vector is defined to generate a torque around the central axis $(x_c, y_c) = (1, 1)$. The load vector $\mathbf{g}$ is specified as:

$$\mathbf{g}(x, y, z, t) = \begin{bmatrix} -10 \cdot (y - 1) \\ +10 \cdot (x - 1) \\ 0 \end{bmatrix} \cdot \sin(t) \cdot \mathbb{I}_{\{z>0.9\}} \tag{30}$$

The term $[-(y-1),(x-1)]$ represents the tangent vector to the circle centered at $(1,1)$. The $\sin(t)$ factor modulates the intensity over time, simulating a cyclic twisting motion.

Results. The simulation successfully reproduced the expected twisting deformation. As illustrated in Figure 6:

- **Kinematics:** The displacement magnitude increases linearly with the vertical coordinate z, consistent with the analytical solution for the torsion of a cylinder (Saint-Venant torsion [19]). The base ($z=0$) remains stationary (blue region), while the top surface ($z=1$) exhibits the maximum rotational displacement (red region).
- **Shear Effects:** The mesh elements undergo significant shear deformation, which in a poroelastic medium induces coupled fluid pressure gradients. The solver correctly captures this interaction without locking or instability.

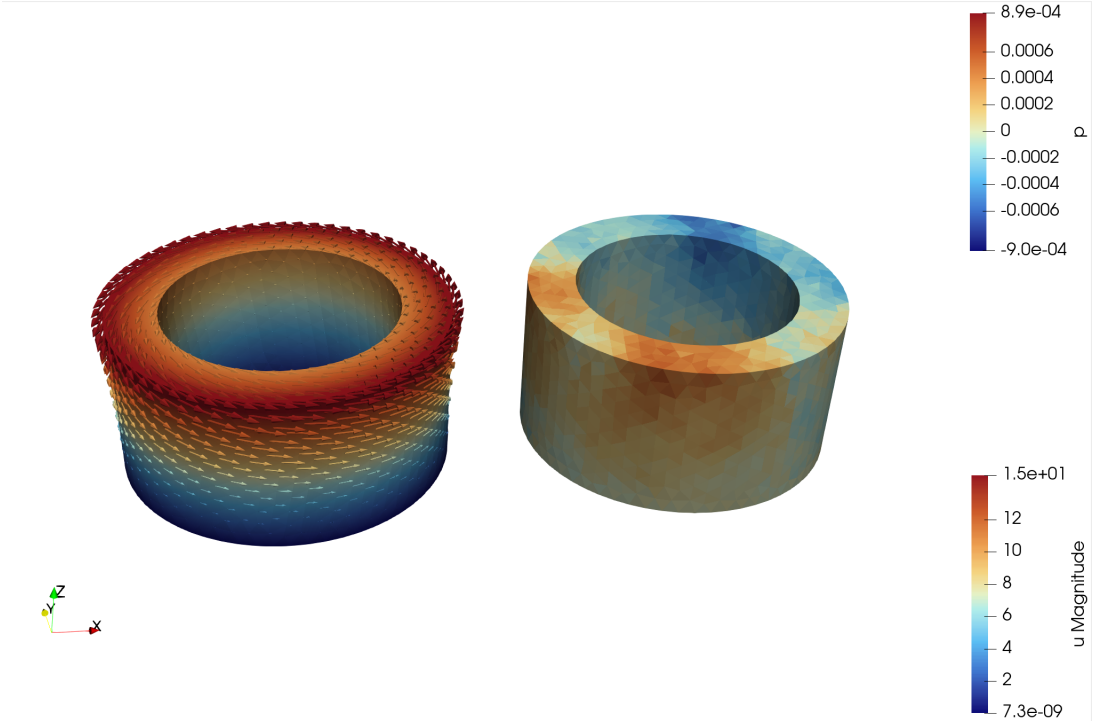


Figure 6: Numerical results for the torsional loading test at $t = 1.8$s. **Left:** Displacement magnitude field [m]. **Right:** Pore pressure distribution [Pa].

This result confirms that the implemented parser correctly interprets vector-valued functions of space coordinates, enabling the simulation of complex physiological loading patterns beyond simple axial compression.

9.5 Application: The "Russian Doll" Multiscale Simulation

To demonstrate the full capability of the developed framework, we applied the "Russian Doll" algorithm to model the interaction between the vascular porosity (PV) and the lacuno-canalicular porosity (PLC) in a bone osteon.

Setup and Coupling Strategy. The simulation setup involves two nested domains:

1. **Macro-Scale (PV):** Represents a cortical bone segment ($R_{ext} \approx 1.0$) subjected to the time-dependent axial compression load defined in Section 9.4.1. The outer wall is fixed.

2. **Data Transfer:** An `InterpolationManager` extracts the pressure p_v and displacement $\mathbf{u}_v$ solutions from the Macro domain along a vertical line located at $(x = 0.25, y = 1.0)$, corresponding to the region of maximum compressive deformation. These discrete data points are fitted to 5th-order polynomials $P_{fit}(z)$ and $\mathbf{U}_{fit}(z)$ using Ordinary Least Squares (OLS) to filter numerical noise and provide a continuous boundary representation.

3. **Micro-Scale (PLC):** Represents the single osteon located at the extraction coordinates. The fitted polynomials are applied as Dirichlet boundary conditions on the outer wall of the PLC mesh ($\mathbf{u}_{PLC}|_{\Gamma_{out}} = \mathbf{U}_{fit}$). Additionally, the macroscopic pressure drives the fluid exchange via the leakage source term $\gamma(p_v - p_l)$. Crucially, no external mechanical loads are applied directly to the PLC; its deformation is driven entirely by the displacement of the surrounding PV matrix.

Results and Physical Consistency. The simulation was executed for 20 time steps ($T_{end} = 2.0$s) using the `BiCGSTAB` solver. The results confirm the effectiveness of the one-way coupling mechanism.

- **Mechanical Coupling:** as illustrated in Figure 7, the PLC domain faithfully reproduces the local deformation of the PV material. The outer wall of the osteon exhibits a vertical displacement gradient consistent with the compression of the macro-segment. This confirms that the polynomial interpolation correctly transferred the Dirichlet boundary conditions, effectively "embedding" the osteon within the deforming cortical bone.

- **Hydraulic Coupling:** the pressure results (Figure 8) demonstrate the dual nature of the poroelastic coupling. The PLC develops a non-trivial pressure distribution. The convergence logs confirm that the coupling norm is non-zero ($\approx 2.88 \cdot 10^{-3}$), indicating active fluid exchange between the vascular and lacunar porosities.

Computational Performance. The hierarchical algorithm proved robust and efficient:

- **Polynomial Fitting:** the OLS procedure (order 5) successfully captured the macroscopic field variations without introducing numerical artifacts. The fitting coefficients (e.g., $P_{coeff} \approx [-2.2 \cdot 10^{-6}, \dots]$) show a smooth variation along the osteon axis.

- **Solver Stability:** despite the high condition number, the linear solver maintained convergence (30 iterations per step for the PLC problem), correctly resolving the small-scale displacements induced by the coupling.

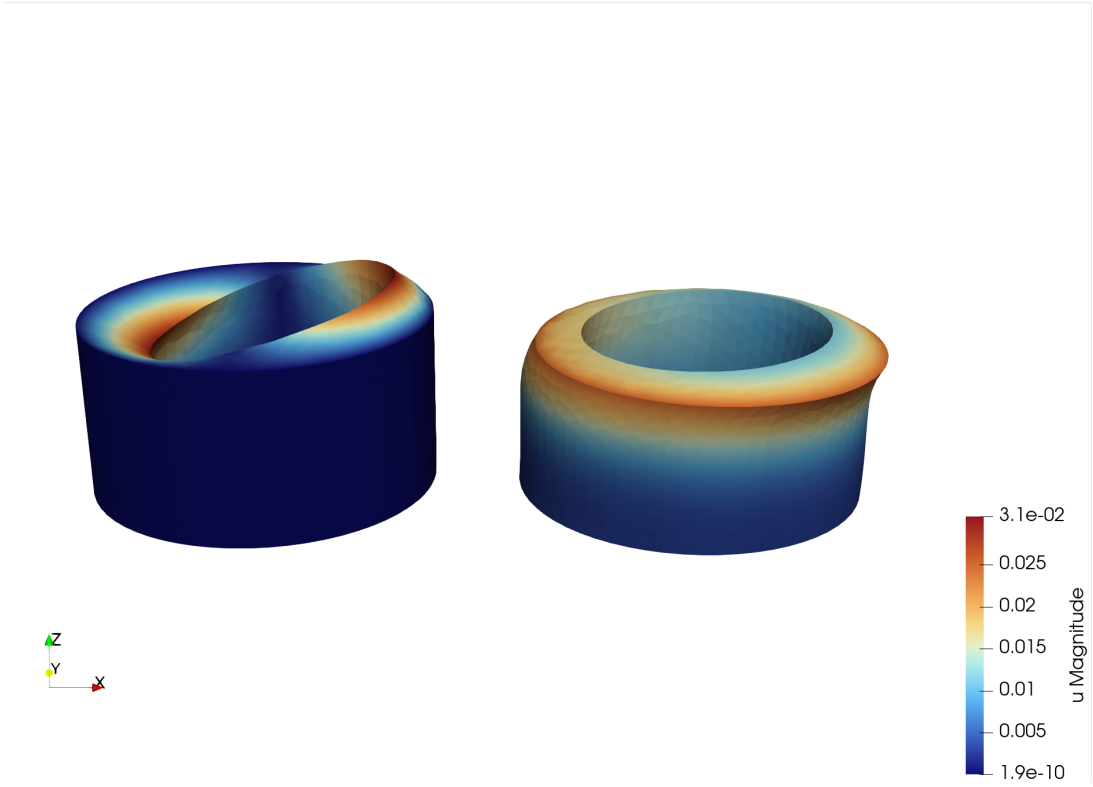


Figure 7: Displacement coupling in the Russian Doll model at $t = 2.0$s. **Left (PV):** The macroscopic cylinder undergoes axial compression. **Right (PLC):** The microscopic osteon, located at the point of maximum compression ($x = 0.25$), inherits the boundary deformation. The outer wall is not stationary but follows the compressive strain of the macro-matrix.

10 Conclusions

10.1 Summary of Achievements

In this project, we successfully designed and implemented a flexible C++ finite element solver for the coupled Biot poroelasticity equations, specifically tailored for multi-scale bone tissue simulation.
From a **computational perspective**, the main achievements are:

- **Modular Architecture:** we developed a robust Object-Oriented code based on `GetFEM++`. By adhering to design patterns such as *Composition* (for the Coupled problem) and *Strategy* (via the input Parser), we achieved a strict separation between physical problem definitions, mesh management, and linear algebra backends.
- **Robust Numerics:** the implementation of Mixed Finite Elements ($\mathbb{RT}_0 - \mathbb{P}_0$) ensures local mass conservation, while the monolithic coupling strategy preserves the strong physical interaction between fluid and solid phases. The solver proved robust in both 2D verification benchmarks and complex 3D scenarios.
- **Multi-Scale Implementation:** we successfully implemented the "Russian Doll" algorithm using a custom `InterpolationManager` to handle non-conforming mesh

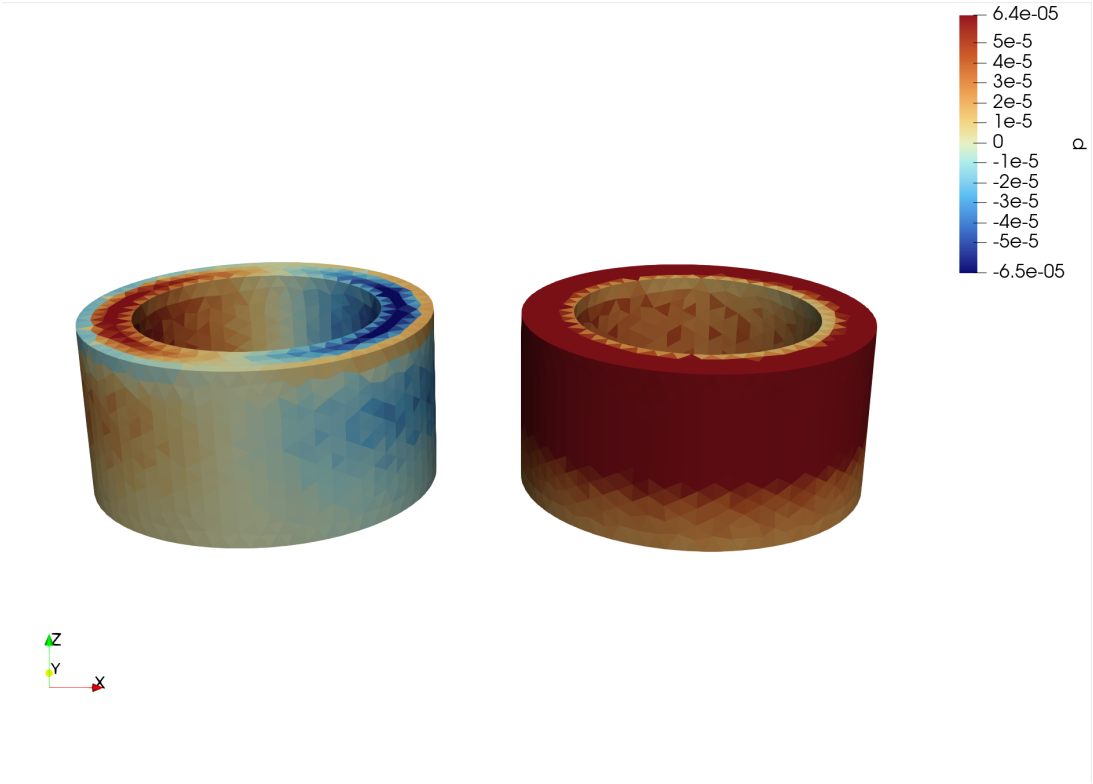


Figure 8: Pressure coupling. **Left (PV):** Macroscopic pressure gradient induced by the poroelastic effect. **Right (PLC):** The osteon develops an internal pressure field driven by two factors: the volumetric strain induced by the mechanical boundary conditions, and the leakage source term driven by the macroscopic pressure p_v.

data transfer via Ordinary Least Squares (OLS) fitting.

- **Advanced Solver Capabilities:** the framework supports both direct solvers (`SuperLU`, and the parallel `MUMPS`) and iterative solvers (`BiCGSTAB`, `GMRES`). The integration of the Generic Weak Form Language (GWFL) allows for runtime modification of material laws and source terms without recompilation.

From a **mechanobiological perspective**, the simulations confirmed that the movement of interstitial fluid within cortical bone is directly driven by external mechanical actions.

- **Coupling Validation:** the results clearly demonstrate that fluid pressure and velocity inside the hierarchical porous network respond to applied loads: when the osteon is compressed or twisted, pressure gradients develop in the vascular porosity (PV) and induce fluid exchange with the lacuno–canalicular porosity (PLC).

10.2 Limitations

Despite the successful verification and application of the code, several limitations regarding both the numerical method and the physical modeling remain:

1. **One-Way Coupling:** the current implementation of the Russian Doll model assumes a one-way coupling (PV $\rightarrow$ PLC). While physically justified by the scale

separation, a full two-way coupling would be required to capture the feedback of the micro-porosity on the macro-scale pressure field.

2. **Modeling of Leakage (γ):** the leakage parameter γ remains uncertain and often requires calibration. Moreover, the linear law $\gamma(p_v - p_l)$ may not capture more complex transport mechanisms such as non-linear permeability.

3. **Geometric Idealization:** the bone is idealized as a cylindrical domain with homogeneous and isotropic material properties. Real cortical bone exhibits heterogeneity, anisotropy, and cement lines that significantly influence local fluid flow [22].

4. **Absence of Biological Feedback:** biological remodeling processes are not explicitly included. Consequently, the framework describes the mechanical and fluid-related stimuli (the "trigger"), but not the resulting change in bone density or architecture (the "outcome").

5. **Computational Scalability:** while the `MUMPS` solver allows utilizing multiple cores for factorization, the lack of distributed-memory parallelism (MPI domain decomposition) limits the maximum problem size to the RAM available on a single device.

10.3 Future Extensions

Future developments will focus on enhancing both computational performance and physical fidelity:

1. **Distributed Parallelization:** interfacing the code with GetFEM's MPI-based domain decomposition tools to enable fully distributed assembly. This is essential to handle realistic patient-specific bone meshes with millions of elements.

2. **Realistic Geometries and Anisotropy:** moving beyond the ideal cylinder to consider patient-specific osteonal geometries derived from micro-CT scans. This includes implementing anisotropic permeability tensors to reflect the structural orientation of the collagen matrix.

3. **Non-linear Solvers:** implementing a fixed-point or Newton-Raphson loop to solve non-linear poroelasticity problems, where permeability is a function of the volumetric strain.

4. **Advanced Preconditioning:** developing block-preconditioners specifically designed for the saddle-point structure of poroelasticity to improve the convergence rate of iterative solvers like GMRES, making them competitive with direct solvers for large-scale problems.

10.4 Lessons Learned

Working on this multiphysics framework was a significant learning experience that went beyond simple coding. We faced several challenges that highlighted the importance of software architecture in scientific computing:

Modularity simplifies debugging: we realized that keeping the physics implementation strictly separated from the low-level mesh management was not just an aesthetic choice, but a practical necessity. This abstraction allowed us to test and fix the Darcy and Elasticity modules individually, making the final coupling phase much smoother.

The power of a Data-Driven approach: implementing a robust input file parser completely changed our workflow. Instead of recompiling the C++ code for every small parameter change, which is time-consuming, we could simply modify a text file. This transformed our code from a static solver into a flexible tool for running multiple experiments quickly.

Balancing high-level libraries and efficiency: using a library like `GetFEM++` allowed us to write complex weak forms in just a few lines of code. However, we learned that relying on high-level tools has a cost: to avoid performance bottlenecks in large 3D simulations, it is still crucial to understand how the library manages memory and linear algebra.

Teamwork requires the right tools: developing this solver as a team taught us that good code organization is crucial. Using Git for version control let us work on different parts of the code at the same time without ruining each other's work. Keeping clear documentation with Doxygen made sure everyone understood how the pieces fit together, and having a reliable Makefile meant we could build and test the code consistently.

10.5 Medical Applications

Beyond the numerical implementation, this project helped us understand how computational models can have a concrete impact on medicine and patient care.

Understanding Osteoporosis: one of the most interesting applications is studying bone loss. Our simulations show how mechanical loads drive the fluid flow that keeps bone cells healthy. This confirms that computational models could be used to design specific physical exercises that help prevent osteoporosis and strengthen bone structure without drugs.

Optimizing Rehabilitation: these models could become predictive tools for doctors. By simulating different loading scenarios, it might be possible to estimate how a fracture will heal under specific rehabilitation protocols, helping to define the best recovery plan for a patient after surgery.

Towards Personalized Medicine: since every bone is unique, the ability to run these simulations on patient-specific geometries (e.g., from CT scans) opens the door to personalized treatments. This could be used to optimize the design of prosthetics or to predict if a specific patient is at risk of fracture under certain loads.

A Class Diagram

Through Doxygen we created a visual map highliting the main dependencies between classes, as one can see in Figure 9.

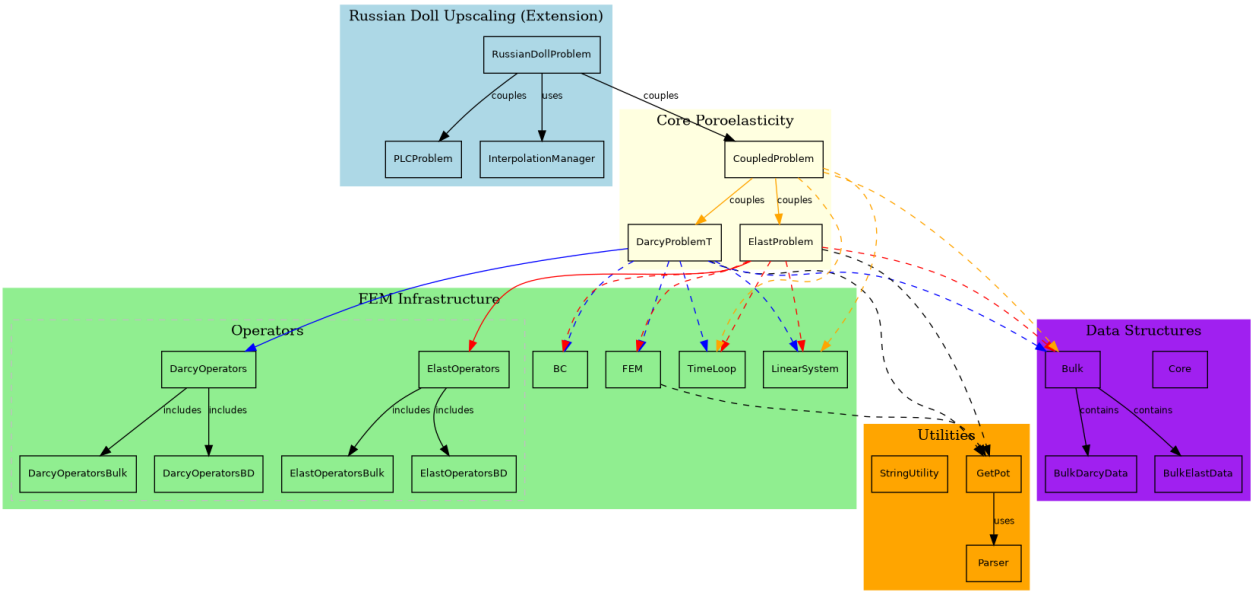


Figure 9: Class diagram generated through Doxygen

B Input File Reference

The simulation is controlled through a plain-text parameter file organized into hierarchical sections. Each section is delimited by square brackets (e.g., `[time]`) and closed with `[../]`. Parameters within sections are specified as `key = value` pairs, with comments indicated by the `%` character.

B.1 Time Discretization

Parameter	Description	Example
`Tend`	Total simulation time	`2`
`dt`	Time step size	`0.1`

B.2 Material Properties

Parameter	Description	Example
`gravity`	Gravitational acceleration vector	`[0,0]`
`rho_r`	Solid matrix density ρ_r	`1`
`rho_f`	Fluid density ρ_f	`1`
`mu_f`	Fluid dynamic viscosity η	`1`

B.3 Domain Configuration

Parameter	Description	Example
`meshExternal`	External mesh filename	`mesh_tagName.msh`
`meshFolder`	Path to mesh directory	`./meshes/`
`spaceDimension`	Spatial dimension (2D/3D)	`3`
`nBoundaries`	Number of boundary regions	`4`
`interpolationDegree`	Polynomial interpolation degree	`5`
`boundaryType`	Boundary identification method	`tagName`
`tagSelection`	Tag selection mode for TagNumber	`largest`

B.4 Darcy Flow Parameters

Parameter	Description	Example
`bcflag`	Boundary condition flags (0=Dirichlet, 1=Neumann)	`[1,1,0,0]`
`Kxx, Kyy, Kzz`	Diagonal permeability tensor components κ_{ii}	`1.0`
`Kxy, Kxz, Kyz`	Off-diagonal permeability components κ_{ij}	`0.0`
`biotM`	Biot modulus M	`1`
`BiotAlpha`	Biot coefficient α	`1`
`leakage`	Leakage coefficient(enable for russian doll)	`1`
`source`	Volumetric source/sink term	`0`
`p_BC`	Pressure boundary condition	`0`
`v_BC`	Velocity boundary condition	`0`
`pIni`	Initial pressure field	`0`

Finite element types for Darcy flow:

- `FEMTypeVelocity`: Velocity space (e.g., `FEM_RT0(3)` for Raviart-Thomas)
- `FEMTypePressure`: Pressure space (e.g., `FEM_PK(3,0)` for piecewise constant)
- `integrationMethod`: Numerical quadrature rule (e.g., `IM_TETRAHEDRON(2)`)

B.5 Mechanics Parameters

Parameter	Description	Example
`bcflag`	BC flags (0=Dirichlet, 1=Neumann)	`[0,1,1,1]`
`mu`	Shear modulus (Lamé parameter μ)	`1`
`lambda`	First Lamé parameter λ	`1`
`fluidP`	Prescribed fluid pressure (decoupled mode)	`0`
`bulkLoad`	Body force vector $\mathbf{F}$	`[0,0,0]`
`bdLoad`	Boundary traction (supports expressions)	`[0,0,0.1*(x-1)*sin(t)*(z>0.58)]`
`bdDisp`	Prescribed boundary displacement	`[0,0,0]`
`bdVel`	Prescribed boundary velocity	`[0,0,0]`
`u_ini`	Initial displacement field	`[0,0,0]`

The boundary load expression `bdLoad = [0,0,0.1*(x-1)*sin(t)*(z>0.58)]` demonstrates the parser's support for spatial coordinates (x, y, z), time t, and conditional expressions.

B.6 Solver Configuration

Parameter	Description	Example
`type`	Linear solver	`BICGSTAB`, `GMRES`, `SUPERLU`, `MUMPS`
`preconditioner`	Preconditioner	`ILU`, `ILUT`, `DIAGONAL`, `NONE`
`maxIterations`	Maximum iterations (iterative solvers)	`1000`
`tolerance`	Convergence tolerance	`1.0e-8`

B.7 Russian Doll Interpolation (Multi-scale Problems)

For nested domain simulations, the `[russian_doll/interpolation]` section defines sampling lines for field transfer:

Parameter	Description	Example
`approach`	Interpolation approach	`line`
`num_lines`	Number of sampling lines	`1`
`start_x/y/z`	Line starting point coordinates	`0.25, 1.0, 0.0`
`end_x/y/z`	Line ending point coordinates	`0.25, 1.0, 1.0`
`num_samples`	Number of sampling points	`100`
`polynomial_order`	Interpolation polynomial order	`5`

C Build Instructions

This section provides instructions for building the poroelasticity solver from source. The software depends on the GetFEM finite element library and can be compiled in either serial or parallel mode (with MUMPS sparse solver support).

C.1 System Requirements

- **Operating System**: Linux (tested on Ubuntu 22.04/24.04)
- **Compiler**: GCC with C++17 support (`g++ >= 7.0`)
- **Build System**: GNU Make

C.2 Step 1: Install System Dependencies

Install the required development libraries using the package manager:

```
# Core dependencies
sudo apt-get update
sudo apt-get install build-essential g++ gfortran

# Linear algebra libraries
sudo apt-get install libopenblas-dev liblapack-dev

# Sparse solvers and graph partitioning
sudo apt-get install libsuperlu-dev libqhull-dev
```

```
sudo apt-get install libscotch-dev libmetis-dev

# For parallel builds (MUMPS + MPI)
sudo apt-get install libopenmpi-dev libmumps-dev

# Additional tools
sudo apt-get install automake libtool
```

C.3 Step 2: Install GetFEM Library

GetFEM must be compiled from source to ensure proper configuration. Two installation paths are recommended: one for serial builds and one for parallel builds with OpenMP and MUMPS support.

C.3.1 Serial Installation

```
# Download GetFEM
cd ~/
mkdir -p getfem && cd getfem
wget https://download.savannah.gnu.org/releases/getfem/stable/getfem-5.4.4.tar.gz
tar -xzf getfem-5.4.4.tar.gz
cd getfem-5.4.4

# Configure for serial build
./configure --prefix=$HOME/getfem/getfem-5.4.4-standard \
            --with-blas=openblas

# Build and install
make -j$(nproc)
make install
```

C.3.2 Parallel Installation (with MUMPS and OpenMP)

```
# Extract fresh copy (same as above)
cd ~/
mkdir -p getfem && cd getfem
wget https://download.savannah.gnu.org/releases/getfem/stable/getfem-5.4.4.tar.gz
tar -xzf getfem-5.4.4.tar.gz
cd getfem-5.4.4

#configure with extra flags
./configure --prefix=$HOME/getfem/getfem-5.4.4-parallel \
            --enable-openmp \
            --enable-mumps \
            --with-blas="-lopenblas" \
            --with-mumps="-ldmumps -lsmumps -lzmumps -lcmumps \
                          -lmumps_common -lpord" \
```

```
            --with-pic \
            --disable-paralevel

# Build and install
make -j$(nproc)
make install
```

The `-enable-openmp` flag enables multi-threaded assembly, while `-enable-mumps` links the MUMPS parallel direct solver for improved performance on large systems.

C.4 Step 3: Build the Poroelasticity Solver

Clone the repository:

```
git clone https://github.com/yourusername/your-repo-name
cd your-repo-name
```

Build the solver using the provided Makefile:

```
# Serial build (default)
make clean
make coupled          # or: make russian

# Parallel build
make clean
make parallel coupled # or: make parallel russian

# Custom GetFEM path (if installed elsewhere)
make GETFEM_PREFIX=/path/to/getfem coupled
```

Available build options:

Option	Description
`PARALLEL=1`	Enable MUMPS solver and OpenMP
`BUILD_TYPE=debug`	Debug symbols, no optimization
`BUILD_TYPE=release`	Full optimization (default)
`VERBOSE_MODE=1`	Enable verbose output

C.5 Step 4: Running the Simulation

```
# Serial execution
./main_coupled -f input/data_ideal_parameter.txt

# Parallel execution (set thread count first)
export OPENBLAS_NUM_THREADS=4
./main_coupled -f input/data_ideal_parameter.txt
```

References

[1] P. R. Amestoy, I. S. Duff, J.-Y. L'Excellent, and J. Koster. A fully asynchronous multifrontal solver using distributed dynamic scheduling. *SIAM Journal on Matrix Analysis and Applications*, 23(1):15–41, 2001. `https://mumps-solver.org/`.

[2] F. Brezzi and M. Fortin. *Mixed and Hybrid Finite Element Methods*. Springer Series in Computational Mathematics. Springer, 1991.

[3] E. H. Burger, J. Klein-Nulend, and T. H. Smit. Strain-derived canalicular fluid flow regulates osteoclast activity in a remodelling osteon — a proposal. *Journal of Biomechanics*, 36:1453–1459, 2003.

[4] L. Cardoso, S. P. Fritton, G. Gailani, M. Benalla, and S. C. Cowin. Advances in assessment of bone porosity, permeability and interstitial fluid flow. *Journal of Biomechanics*, 46(2):253–265, 2013.

[5] Stephen C. Cowin. Bone poroelasticity. *Journal of Biomechanics*, 32(3):217–238, 1999.

[6] Stephen C Cowin. *Bone mechanics handbook*. CRC press, 2001.

[7] Stephen C. Cowin, G. Gailani, and M. Benalla. Hierarchical poroelasticity: movement of interstitial fluid between porosity levels in bones. *Philosophical Transactions of the Royal Society A*, 367(1902):3401–3444, 2009.

[8] Stephen C. Cowin, G. Gailani, and M. Benalla. Hierarchical poroelasticity: movement of interstitial fluid between porosity levels in bones. *Philosophical Transactions of the Royal Society A*, 367(1902):3401–3444, 2009.

[9] Stephen C. Cowin and D. H. Hegedus. Bone remodeling i: theory of adaptive elasticity. *Journal of Elasticity*, 6(3):313–326, 1976.

[10] Susan P. Fritton and Sheldon Weinbaum. Fluid and solute transport in bone: flow-induced mechanotransduction. *Annual Review of Fluid Mechanics*, 41(1):347–374, 2009.

[11] G. Gailani and S. C. Cowin. Ramp loading in russian doll poroelasticity. *Journal of the Mechanics and Physics of Solids*, 59(1):103–120, 2011.

[12] Xiaoye S Li. An overview of superlu: Distributed-memory, shared-memory and sequential solvers. *ACM Transactions on Mathematical Software (TOMS)*, 31(3):302–325, 2005.

[13] Joachim Nitsche. Über ein variationsprinzip zur lösung von dirichlet-problemen bei verwendung von teilräumen. *Abhandlungen aus dem Mathematischen Seminar der Universität Hamburg*, 36(1):9–15, 1971.

[14] Margareta Nordin and Victor H Frankel. *Basic biomechanics of the musculoskeletal system*. Lippincott Williams & Wilkins, 2012.

[15] A. F. Pereira and S. J. Shefelbine. The influence of load repetition in bone mechanotransduction using poroelastic finite-element models: the impact of permeability. *Biomechanics and Modeling in Mechanobiology*, 13(1):215–225, 2014.

[16] P. A. Raviart and J. M. Thomas. A mixed finite element method for second order elliptic problems. In *Mathematical Aspects of the Finite Element Method*, volume 606 of *Lecture Notes in Mathematics*, pages 292–315. Springer, 1977.

[17] Yves Renard and Konstantinos Poulios. Getfem++: A structured c++ library for generic finite element methods. *Advances in Engineering Software*, 145:102816, 2020.

[18] María Teresa Sánchez, María Ángeles Pérez, and José Manuel García-Aznar. The role of fluid flow on bone mechanobiology: mathematical modeling and simulation. *Computational Geosciences*, 25:823–830, 2021. `https://link.springer.com/article/10.1007/s10596-020-09945-6`.

[19] Stephen P Timoshenko and J N Goodier. *Theory of elasticity*. McGraw-Hill, 1970.

[20] Sheldon Weinbaum, Stephen C. Cowin, and You Zeng. A model for the excitation of osteocytes by mechanical loading-induced bone fluid shear stresses. *Journal of Biomechanics*, 27(3):339–360, 1994.

[21] Julius Wolff. *Das Gesetz der Transformation der Knochen*. Hirschwald, Berlin, 1892.

[22] Peng Zeng, Stephen C. Cowin, and Luis Cardoso. Micromechanically based poroelastic modeling of fluid flow in haversian bone. *Journal of Biomechanical Engineering*, 125(1):25–37, 2003.